

编译系统期末复习

问天讲师团——黄梓涵

1.1.编译：将高级语言翻译成汇编语言或机器语言的过程

用高级语言编写的程序经编译后产生的程序叫(B).

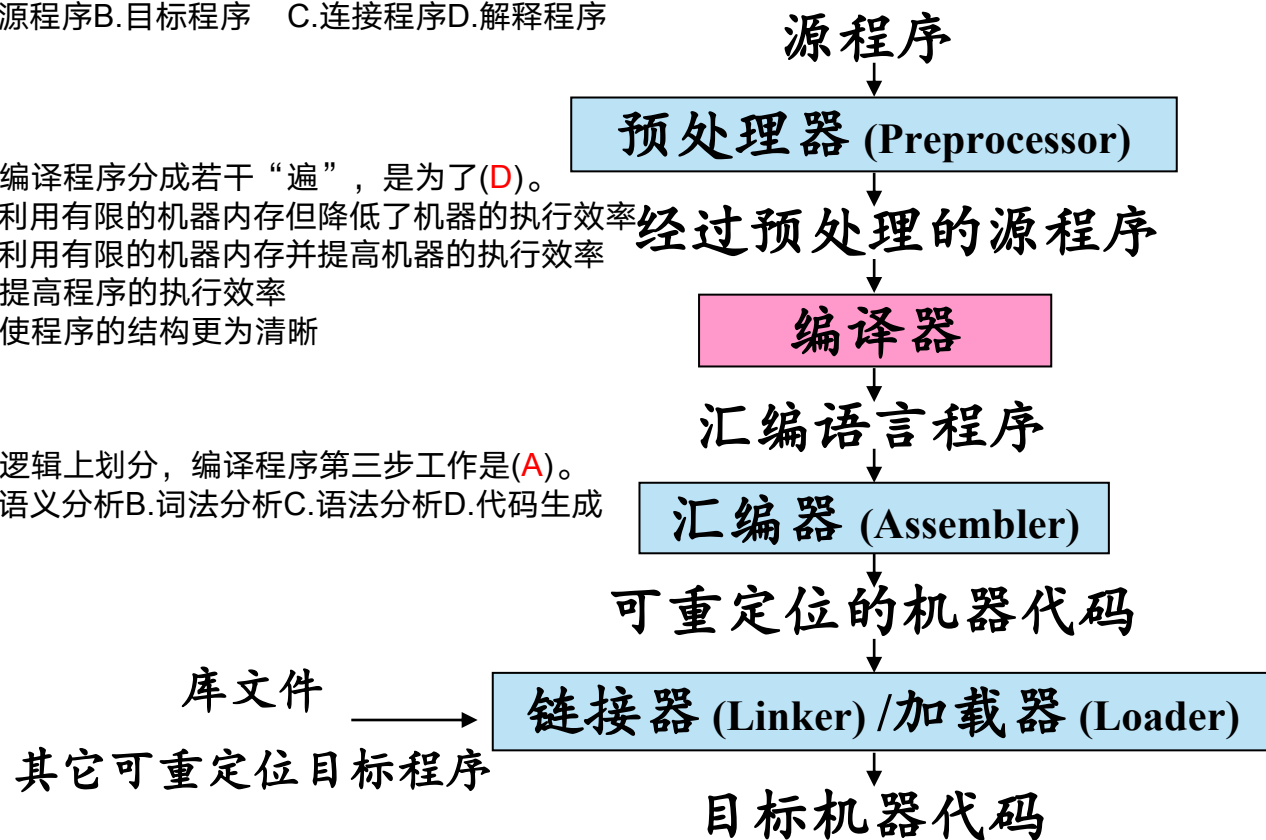
A.源程序 B.目标程序 C.连接程序 D.解释程序

将编译程序分成若干“遍”，是为了(D)。

- A.利用有限的机器内存但降低了机器的执行效率
- B.利用有限的机器内存并提高机器的执行效率
- C.提高程序的执行效率
- D.使程序的结构更为清晰

按逻辑上划分，编译程序第三步工作是(A)。

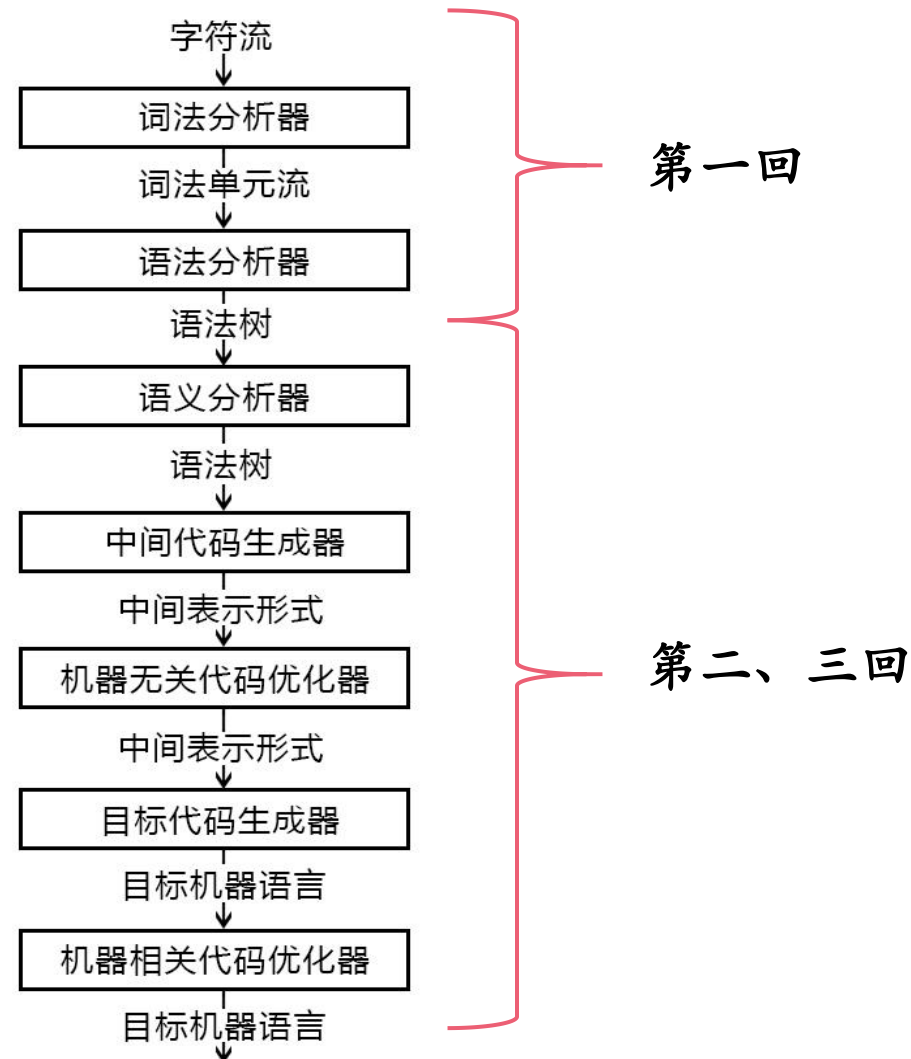
- A.语义分析 B.词法分析 C.语法分析 D.代码生成



通常一个编译程序中，不仅包含词法分析，语法分析，语义分析，中间代码生成，代码优化，目标代码生成等六个部分，还应包括(A)

- A.表格处理和出错处理 B.符号执行器 C.解释器 D.模拟执行器

编译系统的结构



编译过程中，语法分析器的任务就是(B)。

A.分析单词是怎样构成的 B.分析单词串是如何构成语句和声明的

- 词法分析的主要任务：从左向右逐行扫描源程序的字符，识别出各个单词，**确定单词的类型**。将识别出的单词转换成统一的**机内表示**——词法单元(token)形式

单词类型	种别	种别

多词一码只有标识符，可以理解为实验一里面那个ID

	单词类型	种别	种别码
1	关键字	program、if、else、then、...	一词一码
2	标识符	变量名、数组名、记录名、过程名、...	多词一码
3	常量	整型、浮点型、字符型、布尔型、...	一型一码
4	运算符	算术 (+ - * / ++ --) 关系 (> < == != >= <=) 逻辑 (& ~)	一词一码 或 一型一码
5	界限符	； () = { } ...	一词一码

- 输入：token序列， 输出：语法分析树

- (1) 收集标识符的属性信息6种:

种属、类型、存储位置、长度、值、作用域、参数和返回值信息

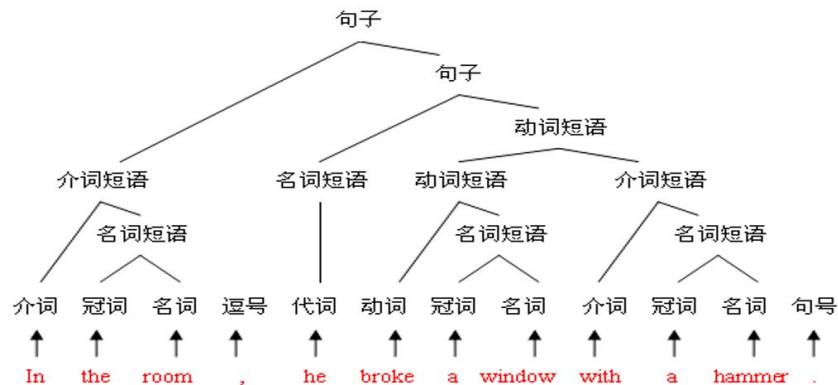
- (2) 语义检查4类:

变量（包括数组、指针、结构体）或过程未经声明就使用

变量（包括数组、指针、结构体）或过程名重复声明

运算分量类型不匹配

操作符与操作数之间的类型不匹配



- 常用的中间表示形式：**三地址码**（四元式、三元式、间接三元式）、**语法结构树**

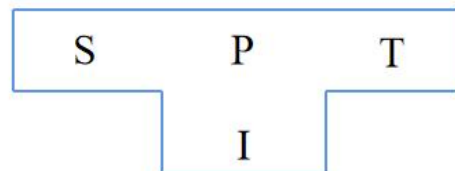
1.3.编译程序的生成

P: 编译器 (程序)

I: 实现语言

S: 输入的源语言程序

T: 输出的可执行的目标程序



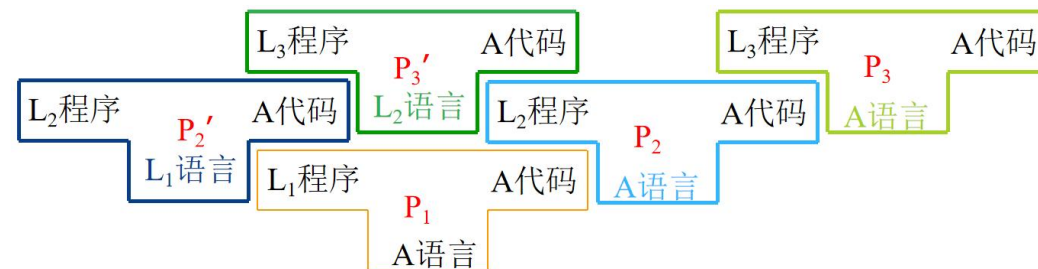
• 自展 (在 **同一台机器** 上实现 **不同语言** 的编译器)

给定P1: A机器上运行的高级语言L1的编译器

构造P2: A机器上运行的高级语言L2的编译器

构造P3: A机器上运行的高级语言L3的编译器

L2比L1更好用, 所以用L2写L3转A的代码, 这样一层层叠上去



• 编译器的移植 (交叉编译): (在 **不同机器** 上实现 **同一语言** 的编译器)

给定P1: A机器上运行的高级语言L的编译器

构造P2: B机器上运行的高级语言L的编译器

1.4.编译技术的应用

结构化编辑器、智能打印机、静态检测器、文本格式器、数

据库查询解释器、高级语言的翻译工具

编译器的自动生成:

LEX: 词法分析程序生成器; YACC: 语法分析程序生成器



2.1.基本概念

文法: $G: S \rightarrow xSx \mid y$ 所识别的语言是(D)。
A. xyx B. $(xyx)^*$ C. x^*yx^* D. $x^nyx^n (n \geq 0)$

文法G产生的(D)的全体是该文法描述的语言。
A. 句型 B. 终结符集 C. 非终结符集 D. 句子

串 (有穷符号的序列)、长度 ($|s|$)、空串 ϵ (长度为0的串)

前缀、后缀、连接 (xy)、幂 (s^n)

设有文法 $G[S]: S \rightarrow S1 \mid S0 \mid Sa \mid Sc \mid a \mid b \mid c$, 下列符号串中是该文法的句子有(D)。
A. $ab0$ B. $a0b01$ C. $a0b0a$ D. $bc10$

字母表 (Σ 是一个有穷符号集合)、字母表的乘积、 n 次幂、正闭包、克林闭包

2.2.文法的定义

约定:

$$G = (V_T, V_N, P, S)$$

终结符集合、非终结符集合、
产生式集合、开始符号

终结符	a, b, c	终结符号串	$u, v, \dots, z$
非终结符	A, B, C		
文法符号	X, Y, Z	文法符号串	α, β, γ

一个上下文无关文法G包括四个组成部分, 它们是一组非终结符号, 一组终结符号, 一个开始符号, 以及一组(B)。
A. 句子 B. 产生式 C. 单词 D. 句型

书写注意: 未用尖括号括起来部分表示语言的基本符号
尖括号括起来部分称为语法成分

$\langle \text{动词短语} \rangle \rightarrow \langle \text{动词} \rangle \langle \text{名词短语} \rangle$

$\langle \text{形容词} \rangle \rightarrow \textit{little}$

2.3.语言的定义

由文法G的开始符号S推导出的所有句子构成的集合称为**文法G生成的语言**, 记为 $L(G)$ 。即

$$L(G) = \{w \mid S \Rightarrow^* w, w \in V_T^*\}$$

语言上的运算: 并、连接、幂、克林闭包、正闭包

其他概念: 句子、句型; **句子是不包含非终结符的句型**

有了文法 (语言规则), 如何判定某一词串是否是该语言的句子?

句子的推导 - 从生成语言的角度

句子的归约 - 从识别语言的角度

2.4.文法的分类

Chomsky 文法分类体系-----**逐级限制、逐级包含**

0型文法：无限制文法 / 短语结构文法： $\alpha \rightarrow \beta$, α 中至少包含1个非终结符

1型文法：上下文有关文法 (**CSG**) : $|\alpha| \leq |\beta|$

2型文法：上下文无关文法 (**CFG**) : $\alpha \in V_N$

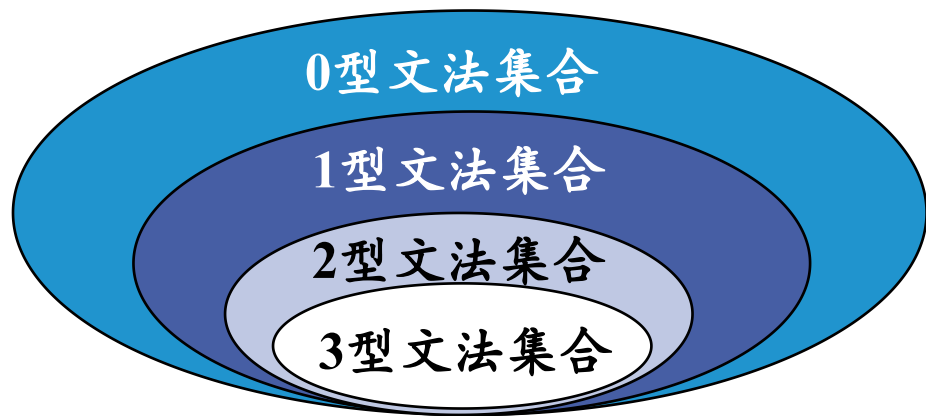
3型文法：正则文法 (**RG**) :

右线性文法 $A \rightarrow wB$ 或 $A \rightarrow w$

左线性文法 $A \rightarrow Bw$ 或 $A \rightarrow w$

若一个文法是递归的，则它所产生的语言的句子(**A**)。

A.是无穷多个 B.是有穷多个 C.是可枚举的 D.个数是常量



2.5.CFG的语法分析树

分析树是推导的**图形化表示**

概念：根节点、内部节点、叶节点、**树的产出（边缘）**、

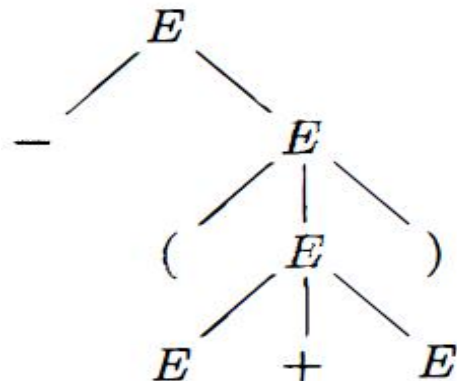
短语、直接短语

直接短语一定是某产生式的右部，但产生式的右部不一定是给定句型的直接短语

二义性文法：如果一个文法可以为某个句子生成多棵分析树，则称这个文法是二义性的。不存在一个算法，判定文法是否为二义性的，但能给出一组**充分条件**。

文法 $E \rightarrow E+E | E^*E | i$ 的句子 $i*i+i*i$ 有 (**C**) 棵不同的语法树。

A.1 B.3 C.5 D.7



3.1.单词的描述

区别：正则语法、正则表达式、正则定义

正则定义：形如 $d_1 \rightarrow r_1$
 $d_2 \rightarrow r_2$
 $\dots$
 $d_n \rightarrow r_n$ 的定义序列

词法分析器的加工对象是 (C)。

A.中间代码 B.单词 C.源程序 D.元程序

词法分析器的输出结果是(D)。

A.单词自身值 B.单词在符号表中的位置
C.单词的种别编码 D.单词的种别编码和自身值

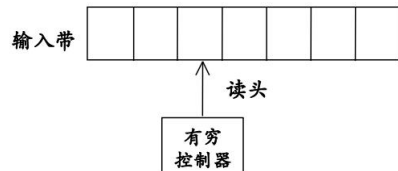
每个 d_i 都是一个**新符号**，它们都不在字母表 Σ 中，而且**各不相同**

词法分析器不能(D)。

A.识别出数值常量 B.过滤源程序中的注释
C.扫描源程序并识别记号 D.发现括号不匹配

每个 r_i 是字母表 $\Sigma \cup \{d_1, d_2, \dots, d_{i-1}\}$ 上的**正则表达式**

3.2.单词的识别



3.2.1有穷自动机

一个正规式只能对应一个确定的有限状态自动机。(X)

一个正规语言可能对应多个正规文法。(√)

FA**模型**：输入带+读头+有穷控制器；FA的**表示**：转换图；FA**定义（接收）的语言**；

最长子串匹配原则：当输入串的多个前缀与一个或多个模式匹配时，总是选择最长的前缀进行匹配

(B)不是DFA的成分。

A.有穷字母表 B.多个初始状态的集合
C.多个终态的集合 D.转换函数

3.2.2有穷自动机的分类

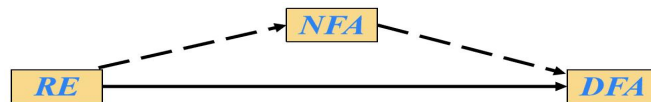
确定的有穷自动机DFA： $M = (S, \Sigma, \delta, s_0, F)$ ，有穷状态集、输入字母表、 **$S \times \Sigma$**

映射到S转换函数、开始状态、接收状态集合

非确定的有穷自动机NFA： $M = (S, \Sigma, \delta, s_0, F)$ ，**将 $S \times \Sigma$ 映射到 2^S 的转换函数**

带有“ ϵ -边”的 NFA： $M = (S, \Sigma, \delta, s_0, F)$ ，**将 $S \times (\Sigma \cup \{\epsilon\})$ 映射到 2^S 的转换函数**

3.2.3从正则表达式到有限自动机



从RE到NFA的转换：分三种情况 $r=r_1|r_2$, $r=r_1r_2$, $r=(r_1)^*$ 构造 ϵ -NFA

从NFA到DFA的转换：子集构造法

RE、NFA、DFA三者识别的语言是等价的

两个有限自动机等价是指它们的(C)。

A.状态数相等 B.有向弧数相等

C.所识别的语言相等 D.状态数和有向弧数相等

3.2.4识别单词的DFA：eg.识别标识符/无符号数/各进制无符号整数/注释/Token的DFA

3.3.词法分析阶段的错误处理

词法错误的类型：单词拼写错误、非法字符

已知文法 $G[S]: S \rightarrow A1, A \rightarrow A1|S0|0$ ，与G等价的正规式是(C)。

A. $0(0|1)^*$ B. $1^*|0^*1$ C. $0(1|10)^*1$ D. $1(10|01^*0)$

词法错误检测：如果当前状态与当前输入符号在转换表对应项中的信息为空，而当前状态又不是终止状态，则调用错误处理程序

错误处理：查找已扫描字符串中最后一个对应于某终态的字符。

如果找到了，将该字符与其前面的字符识别成一个单词，然后将输入指针退回到该字符，扫描器重新回到初始状态，继续识别下一个单词。（并没有删除多出来的字符，只是回退后重新开始）

如果没找到，则确定出错，采用错误恢复策略

错误恢复策略：“恐慌模式”恢复：从剩余的输入中不断删除字符，直到词法分析器能够在剩余输入的开头发现一个正确的字符为止（删除即将读入的）

3.4.词法分析器生成工具Lex Lex语言+Lex编译器

Lex的程序结构：声明部分+转换规则+辅助部分

Lex的构成 (1) 加快了分析器的实现速度 (2) 使得修改扫描器的工作变得更加简单

4. 语法分析

主要任务：根据给定的文法，识别输入句子的各个成分，从而构造出句子的分析树

4.1 自顶向下的分析

最左推导和最右推导具有唯一性 (✓)

在**最左推导**中，总是选择每个句型的最左非终结符进行替换；如果 $S \Rightarrow_{lm}^* \alpha$ ，则称 α 是当前文法的**最左句型**

在**最右推导**中，总是选择每个句型的最右非终结符进行替换；如果 $S \Rightarrow_{rm}^* \alpha$ ，则称 α 是当前文法的**最右句型 (规范句型)**

在自底向上的分析中，总是采用最左归约的方式，因此把**最左归约**称为**规范归约**，而**最右推导**相应地称为**规范推导**

自顶向下语法分析的**通用形式：递归下降分析**(采用最左推导的方式)

由一组**过程**组成，每个过程对应文法的一个**非终结符**

从文法**开始符号S**对应的过程开始，其中**递归调用**文法中其它**非终结符**对应的过程。如果S对应的过程体恰好扫描了**整个输入串**，则成功完成语法分析

如果文法G是无二义的，则它的任何句子 $\alpha(A)$ 。

A. 最左推导和最右推导对应的语法树必定相同

B. 最左推导和最右推导对应的语法树可能不同

C. 最左推导和最右推导必定相同

D. 可能存在两个不同的最左推导，但它们对应的语法树相同

问题1：同一非终结符的多个候选式存在共同前缀，将导致**回溯现象**

递归下降分析需要回溯，除非采用预测分析法（其中的特例）

问题2：左递归文法会使递归下降分析器陷入**无限循环**

解决步骤：

消除间接左递归：

1) 按照某个顺序将非终结符号排序为 $A_1, A_2, \dots, A_n$.

2) for (从1到n的每个*i*) {

3) for (从1到*i*-1的每个*j*) {

4) 将每个形如 $A_i \rightarrow A_j \gamma$ 的产生式**替换**为产生式组

$A_i \rightarrow \delta_1 \gamma \mid \delta_2 \gamma \mid \dots \mid \delta_k \gamma$,

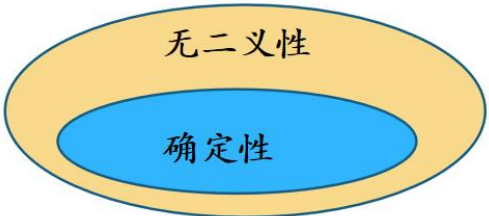
其中 $A_j \rightarrow \delta_1 \mid \delta_2 \mid \dots \mid \delta_k$ ，是所有的 A_j 产生式}

5) 消除 A_i 产生式之间的**直接左递归**}

(1) **消除直接左递归** (把左递归转换成了右递归，代价是引进了一些非终结符和 ϵ -产生式)

$A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \dots \mid A\alpha_n \mid \beta_1 \mid \beta_2 \mid \dots \mid \beta_m$ 变为 $A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \dots \mid \beta_m A'$ 以及 $A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \dots \mid \alpha_n A' \mid \epsilon$

(2) **消除间接左递归** (3) 优化：**提取左公因子** (通过改写产生式来推迟决定，等读入了足够多的输入，获得足够信息后再做出正确的选择) $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2 \mid \dots \mid \alpha\beta_n \mid \gamma_1 \mid \gamma_2 \mid \dots \mid \gamma_m$ 变为 $A \rightarrow \alpha A' \mid \gamma_1 \mid \gamma_2 \mid \dots \mid \gamma_m$ 以及 $A' \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_n$



4.2预测分析法(确定的自顶向下分析)

预测分析法实现步骤

- 1) 构造文法
- 2) 改造文法：消除二义性（满足充分条件）、消除左递归（参考4.1）、消除回溯（例如采用LL(1)）
- 3) 求每个变量的FIRST集和FOLLOW集，从而求得每个候选式的SELECT集
- 4) 检查是不是LL(1)文法。若是，构造预测分析表
- 5) 对于递归的预测分析，根据预测分析表为每一个非终结符编写一个过程；对于非递归的预测分析，实现表驱动的分析算法

工作过程：文法开始符号出发，在每一步推导过程中根据当前句型的最左非终结符A和当前输入符号a，选择正确的A-产生式。为保证分析的确定性，选出的候选式必须是唯一的

可以对某些文法构造出向前看k个输入符号的预测分析器，该类文法有时也称为LL(k)文法类
S_文法:每个产生式的右部都以终结符开始,同一非终结符的各个候选式的首终结符都不同 S_文法不含ε产生式

使用ε产生式的条件：如果当前某非终结符A与当前输入符a不匹配时，若存在A→ε，可以通过检查a是否可以出现在A的后面，来决定是否使用产生式A→ε q_文法不含右部以非终结符打头的产生式

q_文法:每个产生式的右部或为ε，或以终结符开始具有相同左部的产生式有不相交的可选集

在递归子程序方法中，若文法存在左递归，则会使分析过程产生(D)。

A.回溯 B.非法调用
C.有限次调用 D.无限循环

采用自上而下分析，不必(C)。

A.消除回溯 B.消除左递归
C.消除右递归 D.提取公共左因子

在自上而下的语法分析中，应从(C)开始分析。

A.句型 B.句子
C.文法开始符号 D.句柄

已知文法G[S]:
S→eT|RT T→DR|ε R→dR|ε D→a|bd
求FOLLOW(D)= (C) 。

A.{d,e} B.{d,ε}
C.{d,\$} D.{a,d}

已知文法G[S]:
S→eT|RT T→DR|ε R→dR|ε D→a|bd
求FIRST(S)= (D) 。

A.{e} B.{e, d, a, b}
C.{e, d} D.{e, d, a, b, ε}

在语法分析处理中，FIRST集合、FOLLOW集合均是(B)。

A.非终结符集 B.终结符集
C.字母表 D.状态集

FIRST(α)中可以含有 ϵ ，不含\$

FOLLOW(A)中不含 ϵ ，可以含有\$

SELECT($A \rightarrow \beta$)中不含 ϵ ，可以含有\$

4.2.1.LL(1)文法

FIRST(α): 文法符号串 $\alpha = X_1 X_2 \dots X_n$ 的串首终结符集;

定义: ①对于 $\forall \alpha \in (V_T \cup V_N)^+$, $\text{FIRST}(\alpha) = \{a \mid \alpha \Rightarrow^* a\beta, a \in V_T, \beta \in (V_T \cup V_N)^*\}$; ②如果 $\alpha \Rightarrow^* \epsilon$, 那么 $\epsilon \in \text{FIRST}(\alpha)$

计算 (文法符号X) (自底向上实现): ①如果X是一个终结符, 那么 $\text{FIRST}(X) = \{X\}$

②如果X是一个非终结符, 且 $X \rightarrow Y_1 \dots Y_k \in P$ ($k \geq 1$), 对所有i, 若 $a \in \text{FIRST}(Y_i)$ 且 $Y_1 \dots Y_{i-1} \Rightarrow^* \epsilon$, 那么将a加入到 $\text{FIRST}(X)$ 中。若 $Y_1 \dots Y_k \Rightarrow^* \epsilon$ 那么将 ϵ 加入到 $\text{FIRST}(X)$ ③如果 $X \rightarrow \epsilon \in P$, 那么将 ϵ 加入到 $\text{FIRST}(X)$ 中。

计算 (文法符号串 α): ①向 $\text{FIRST}(X_1 X_2 \dots X_n)$ 加入 $\text{FIRST}(X_1)$ 中所有的非 ϵ 符号

②如果 ϵ 在 $\text{FIRST}(X_{i-1})$ 中, 则加入 $\text{FIRST}(X_i)$ 中的所有非 ϵ 符号, 不断执行直到 ϵ 不在 $\text{FIRST}(X_{i-1})$ 中;

③最后, 如果对所有的i, ϵ 都在 $\text{FIRST}(X_i)$ 中, 那么将 ϵ 加入到 $\text{FIRST}(X_1 X_2 \dots X_n)$ 中

FOLLOW(A): 非终结符A的后继符号集; **定义**: ① $\text{FOLLOW}(A) = \{a \mid S \Rightarrow^* \alpha A a \beta, a \in V_T, \alpha, \beta \in (V_T \cup V_N)^*\}$

②如果A是某个句型的的最右符号, 则将结束符“\$”添加到 $\text{FOLLOW}(A)$ 中

计算(自顶向下实现): ①将\$放入 $\text{FOLLOW}(S)$ 中, 其中S是开始符号, \$是输入右端的结束标记。

②若有产生式 $A \rightarrow \alpha B \beta$, 那么 $\text{FIRST}(\beta)$ 中除 ϵ 之外的所有符号都在 $\text{FOLLOW}(B)$ 中。

③若有产生式 $A \rightarrow \alpha B$, 或有 $A \rightarrow \alpha B \beta$ 且 $\epsilon \in \text{FIRST}(\beta)$, 那么 $\text{FOLLOW}(A)$ 中的所有符号都在 $\text{FOLLOW}(B)$ 中

SELECT($A \rightarrow \beta$): 产生式 $A \rightarrow \beta$ 的可选集; **定义**: ① $\text{SELECT}(A \rightarrow a\beta) = \{a\}$; ② $\text{SELECT}(A \rightarrow \epsilon) = \text{FOLLOW}(A)$

计算: ①如果 $\epsilon \notin \text{FIRST}(\alpha)$, 那么 $\text{SELECT}(A \rightarrow \alpha) = \text{FIRST}(\alpha)$

LL(1)文法： 同一非终结符A的各个产生式的可选集SELECT(A→β)互不相交

一个文法G，若(C)，则称它是LL(1)文法。
A.G中不含左递归 B.G无二义性
C.G的LL(1)分析表中不含多重定义条目
D.G中产生式不含左公因子

构造预测分析表：

(1) 首先求出每个非终结符的串首终结符集和后续符号集

X	FIRST(X)	FOLLOW(X)
E	(id	\$)
E'	+ ε	\$)
T	(id	+) \$
T'	* ε	+) \$
F	(id	* +) \$

(2) 使用第一步的信息求出每个产生式的文法符号串的串首终结符集从而求出可选集

	产生式	SELECT
E	E→TE'	(id
E'	E'→+TE'	+
	E'→ε	\$)
T	T→FT'	(id
T'	T'→*FT'	*
F	F→(E)	(
	F→id	id

(3) 根据可选集的表填写预测分析表

非终结符	输入符号					
	id	+	*	(	)	\$
E	E→TE'			E→TE'		
E'		E'→+TE'			E'→ε	E'→ε
T	T→FT'			T→FT'		
T'		T'→ε	T'→*FT'		T'→ε	T'→ε
F	F→id			F→(E)		

(D)文法不是LL(1)的。
A.递归 B.右递归
C.2型 D.含有公共左因子的

4.2.2.递归的预测分析方法：基于预测分析表对递归下降分析法进行扩展

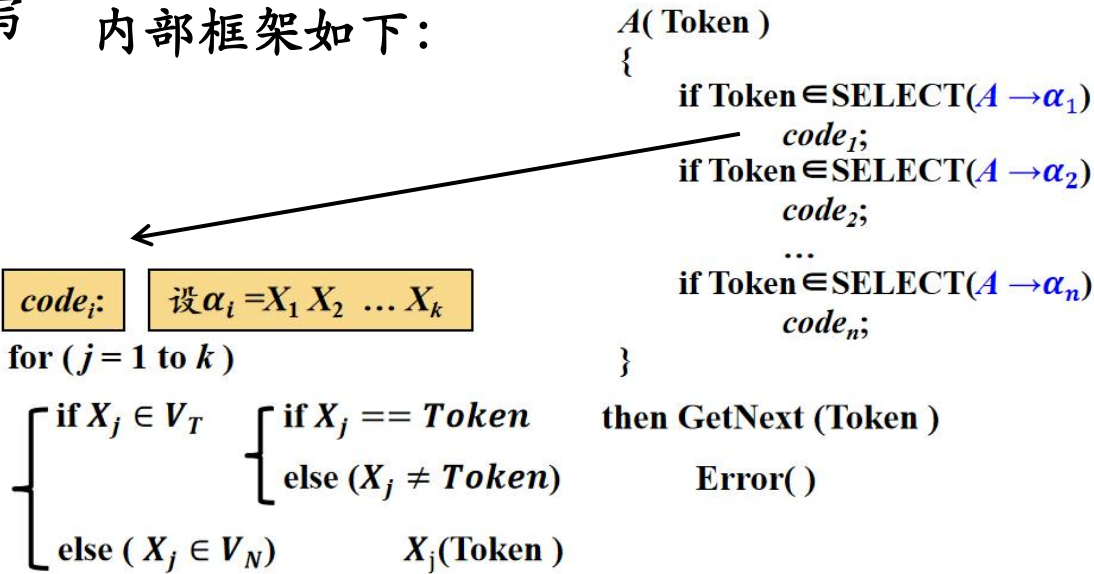
非终结符函数： eg.

<DECLISTN> → , id <DECLISTN>
<DECLISTN>→ε (对应select为{:})

主函数： program类型函数，其中PROGRAM为文法的开始符号，其他书写固定

```
program DESCENT;  
begin  
  GETNEXT(TOKEN);  
  PROGRAM(TOKEN);  
  if TOKEN≠'$' then ERROR;  
end
```

非终结符函数： 每个产生式对应一个procedure类型的函数，对于产生式A → α₁ | α₂ | ... | α_n函数内部框架如下：



```
procedure DECLISTN(TOKEN);  
begin  
  if TOKEN = ',' then  
    begin  
      GETNEXT(TOKEN);  
      if TOKEN ≠ 'id' then ERROR;  
      GETNEXT(TOKEN);  
      DECLISTN(TOKEN);  
    end  
  else if TOKEN ≠ ':' then ERROR;  
end
```

4.2.3.非递归的预测分析方法：显式地维护一个栈结构来模拟最左推导过程

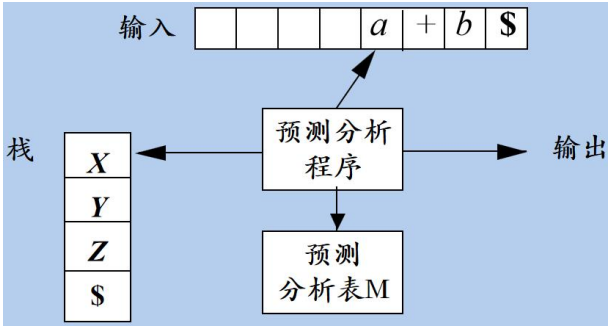
语法分析器的最初格局：输入缓冲区中是串w\$，文法G的开始符号位于栈顶，其下面是\$。

LL (1) 分析法中“1”的含义是在输入串中查看一个输入符号，其目的是（C）。
A.确定最左推导 B.确定句柄
C.确定使用哪一个产生式进行展开 D.确定是否推导

```
设置ip使它指向w的第一个符号，其中ip 是输入指针；
令X=栈顶符号；
while ( X ≠ $ ) { /* 栈非空 */
    if ( X 等于ip所指向的符号a ) 执行栈的弹出操作，将ip向前移动一个位置；
    else if ( X是一个终结符号 ) error ();
    else if ( M[X, a]是一个报错条目 ) error ();
    else if ( M[X, a] = X → Y1Y2... Yk ) {
        输出产生式 X → Y1Y2... Yk ;
        弹出栈顶符号；
        将Yk, Yk-1 ..., Y1 压入栈中，其中Y1位于栈顶。
    }
    令X=栈顶符号
}
```

	递归的预测分析法	非递归的预测分析法
程序规模	程序规模较大，不需载入分析表	主控程序规模较小，需载入分析表（表较小）😄
直观性	较好 😄	较差
效率	较低	分析时间大约正比于待分析程序的长度 😄
自动生成	较难	较易 😄

结构：



4.2.3.预测分析中的错误检测

两种检测可能：①栈顶终结符和当前输入符号不匹配
②栈顶非终结符与当前输入符号在预测分析表对应项中的信息为空
错误恢复：恐慌模式：加入同步词法单元synch(根据相应非终结符的FOLLOW集得到)

非终结符	输入符号					
	id	+	*	(	)	\$
E	E→TE'			E→TE'	synch	synch
E'		E'→+TE'			E'→ε	E'→ε
T	T→FT'	synch		T→FT'	synch	synch
T'		T'→ε	T'→*FT'		T'→ε	T'→ε
F	F→id	synch	synch	F→(E)	synch	synch

- ①如果M[A,a]是空，表示检测到错误，根据恐慌模式，忽略输入符号a
- ②如果M[A,a]是synch，则弹出栈顶的非终结符A，试图继续分析后面的语法成分
- ③如果栈顶的终结符和输入符号不匹配，则弹出栈顶的终结符

在编译过程中，如果遇到错误应该(C)。
A.把错误理解成局部的错误
B.对错误在局部范围内进行纠正，继续向下分析
C.当发现错误时，跳过错误所在的语法单位继续分析下去
D.当发现错误时立即停止编译，待用户改正错误后再继续编译

4.3.自底向上分析

子树中最左边的那棵只有父子两代的子树的所有叶结点自左至右排列起来,就是该句型的句柄(✓)

- (1) $\langle S \rangle \rightarrow \text{var } \langle IDS \rangle : \langle T \rangle$
(2) $\langle IDS \rangle \rightarrow i$
(3) $\langle IDS \rangle \rightarrow \langle IDS \rangle , i$
(4) $\langle T \rangle \rightarrow \text{real} \mid \text{int}$

栈

\$
\$ var
\$ var i_A
\$ var $\langle IDS \rangle$
\$ var $\langle IDS \rangle ,$
\$ var $\langle IDS \rangle , i_B$
\$ var $\langle IDS \rangle , \langle IDS \rangle$
\$ var $\langle IDS \rangle , \langle IDS \rangle :$
\$ var $\langle IDS \rangle , \langle IDS \rangle : \text{real}$
\$ var $\langle IDS \rangle , \langle IDS \rangle : \langle T \rangle$

剩余输入

var $i_A, i_B : \text{real}$ \$
 $i_A, i_B : \text{real}$ \$
 $i_B : \text{real}$ \$
 $i_B : \text{real}$ \$
 $i_B : \text{real}$ \$
 $: \text{real}$ \$
 $: \text{real}$ \$
 $: \text{real}$ \$
 $: \text{real}$ \$
 $: \text{real}$ \$

动作

移入
移入
归约
移入
移入
归约
移入
移入
移入
归约

可以看成是将输入串w归约为文法开始符号S的过程

采用最左归约方式(相当于反向构造句子的最右推导)

通用框架: 移入-归约分析: 维持一个栈一个剩余输入的序列

四种操作: 移入、规约、接收、报错

存在的问题: ①归约-归约冲突 ②移入-规约冲突

句柄: 句型的最左直接短语

在规范归约中, 用(B)来刻画可归约串。
A. 直接短语 B. 句柄
C. 最左素短语 D. 素短语

在自底向上的语法分析方法中, 分析的关键是(A)。
A. 寻找句柄 B. 寻找句型
C. 消除递归 D. 选择候选式

在自顶向下的语法分析方法中, 分析的关键是(D)。
A. 寻找句柄 B. 寻找句型
C. 消除递归 D. 选择候选式

4.4.LR分析法

LR文法是最大的、可以构造出相应移入-归约语法分析器的文法类
L: 对输入进行从左到右的扫描; R: 反向构造出一个最右推导序列

LR(k)分析: 决定是否归约时, 需要向前查看k个输入符号的LR分析

关键问题: 如何正确地识别句柄

句柄是逐步形成的, 用“状态”表示句柄识别的进展程度

LR分析器(自动机)的总体结构

工作过程, 四种情况:

①ACTION[s_m, a_i]=sx: 移入状态 a_i 设置状态x

②ACTION[s_m, a_i]=rx: 第x个产生式归约得到A

GOTO[s_{m-k}, A]=y: 给符号A设置状态y

③ACTION[s_m, a_i]=acc: 分析成功

④ACTION[s_m, a_i]=err: 分析错误

下列动作中, 不是自下而上分析动作的是(B)。
A. 移进 B. 展开
C. 接受 D. 报错

如何构造给定文法的LR分析表:

LR(0), SLR分析, LR(1), LALR分析

下列动作中, 不是自上而下分析动作的是(C)。
A. 匹配 B. 展开
C. 移进 D. 报错

$s_0 s_1 \dots s_m$ $\$ X_1 \dots X_m$					
$a_i a_{i+1} \dots a_n \$$					
状态	ACTION			GOTO	
	a	b	\$	S	B
0	s3	s4		1	2
1			acc		
2	s3	s4			5
3	s3	s4			6
4	r3	r3	r3		
5	r1	r1	r1		
6	r2	r2	r2		

分析栈中内容+剩余输入符号=规范句型

规范句型的活前缀: 不含句柄右侧任意符号的规范句型的前缀

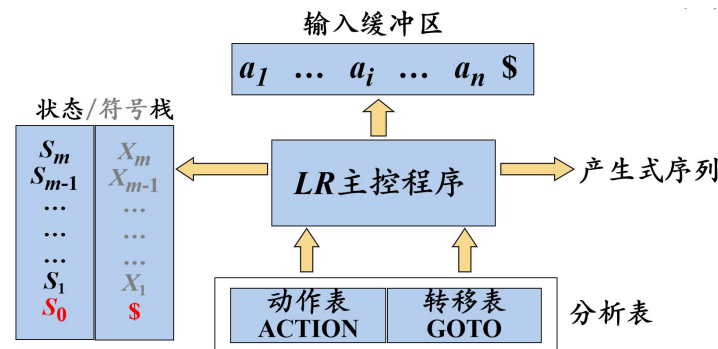
例: $S \rightarrow bBB$

$\triangleright S \rightarrow \cdot bBB$ 移进状态

$\triangleright S \rightarrow b \cdot BB$ 待约状态

$\triangleright S \rightarrow bB \cdot B$ 待约状态

$\triangleright S \rightarrow bBB \cdot$ 归约状态



4.4.1.LR(0)分析

$A \rightarrow \epsilon$ 只生成一个项目 $A \rightarrow \cdot$

例： $S \rightarrow bBB$

若a为终结符，则 $A \rightarrow \alpha \cdot a \beta$ 为(B)项目。
A.归约 B.移进 C.接受 D.待约

若B为非终结符，则 $A \rightarrow \alpha \cdot B \beta$ 为(D)。
A.移进项目 B.归约项目
C.接受项目 D.待约项目

若A为非终结符，则 $A \rightarrow \alpha \cdot$ 为(A)项目。
A.归约 B.移进 C.接受 D.待约

下列项目中为可归约项目的是 (B)
A. $E' \rightarrow \cdot E$ B. $L \rightarrow \cdot$
C. $L \rightarrow \cdot L$ D. $F \rightarrow L^* \cdot F$

LR(0)项目 (简称为项目)： $A \rightarrow \alpha_1 \cdot \alpha_2$

初始项目： $S' \rightarrow \cdot S$ ，接收项目： $S' \rightarrow S \cdot$

移进项目：后继符号为终结符的项目称为移进项目

待约项目：后继符号为非终结符的项目，称为待约项目。

归约项目：后继符号为空的项目称为归约项目

后继项目： $A \rightarrow \alpha \cdot X \beta$ 的后继项目是 $A \rightarrow \alpha X \cdot \beta$

增广文法：在G中加上新开始符号 S' 和产生式 $S' \rightarrow S$

得到的文法,这使得分析器只有一个接受状态

➤ $S \rightarrow \cdot bBB$ ← 移进项目
➤ $S \rightarrow b \cdot BB$ } 待约项目
➤ $S \rightarrow bB \cdot B$ }
➤ $S \rightarrow bBB \cdot$ ← 归约项目

① $S' \rightarrow S$ ② $S \rightarrow vI:T$ ③ $I \rightarrow I,i$ ④ $I \rightarrow i$ ⑤ $T \rightarrow r$

(2) $S \rightarrow \cdot vI:T$
(3) $S \rightarrow v \cdot I:T$ (7) $I \rightarrow \cdot I,i$
(4) $S \rightarrow vI \cdot :T$ (8) $I \rightarrow I \cdot ,i$
(0) $S' \rightarrow \cdot S$ (5) $S \rightarrow vI \cdot :T$ (9) $I \rightarrow I \cdot ,i$ (11) $I \rightarrow \cdot i$ (13) $T \rightarrow \cdot r$
(1) $S' \rightarrow S \cdot$ (6) $S \rightarrow vI:T \cdot$ (10) $I \rightarrow I,i \cdot$ (12) $I \rightarrow i \cdot$ (14) $T \rightarrow r \cdot$

项目集闭包：把等价的项目组成的一个项目集(I)，每个项目集闭包对应着自动机的一个状态

$CLOSURE(I) = I \cup \{B \rightarrow \cdot \gamma \mid A \rightarrow \alpha \cdot B \beta \in CLOSURE(I), B \rightarrow \gamma \in P\}$

GOTO函数返回项目集I对应于文法符号X的后继项目集闭包

$GOTO(I, X) = CLOSURE(\{A \rightarrow \alpha X \cdot \beta \mid A \rightarrow \alpha \cdot X \beta \in I\})$

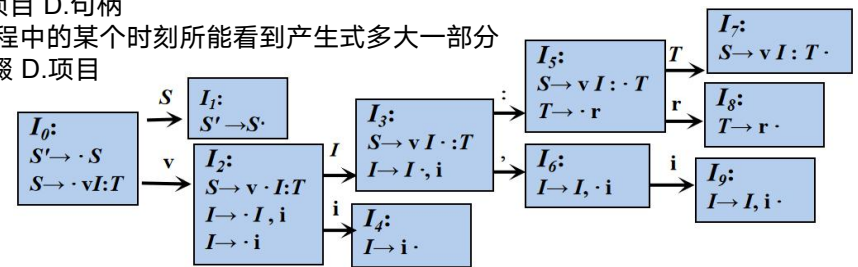
(这个GOTO函数不是LR分析表里的GOTO项)

规范LR(0)项集族

$C = \{I_0\} \cup \{I \mid \exists J \in C, X \in V_N \cup V_T, I = GOTO(J, X)\}$

在LR分析中，分析栈中存放的状态是识别规范句型 (B) 的DFA状态
A.前缀 B.活前缀 C.LR (0) 项目 D.句柄
一个 (B) 指明了在LR分析过程中的某个时刻所能看到产生式多大一部分
A.前缀 B.活前缀 C.归约活前缀 D.项目

LR自动机中从初始状态开始的每一条路径对应一个规范句型的活前缀



LR(0)分析表构造算法：

➤ $CLOSURE(\{[S' \rightarrow \cdot S]\}) \rightarrow C$

➤ for each $I_i \in C$

➤ if $A \rightarrow \alpha \cdot a \beta \in I_i$: 令 $I_j = GOTO(I_i, a)$; $ACTION[i, a] = s_j$

➤ if $A \rightarrow \alpha \cdot B \beta \in I_i$: 令 $I_j = GOTO(I_i, B)$; $GOTO[i, B] = j$

➤ if $A \rightarrow \alpha \cdot \in I_i$: $\left\{ \begin{array}{l} \text{if } A = S' (S' \rightarrow S \cdot \in I_i) \text{ then } ACTION[i, \$] = acc \\ \text{if } A \neq S' \text{ for } \forall a \in V_T \cup \{\$, \} \text{ do } ACTION[i, a] = r_j \end{array} \right.$ (j 是产生式 $A \rightarrow \alpha$ 的编号)

➤ 没有定义的所有条目都设置为“error”

设有文法G[T]:
 $T \rightarrow T^*F|F$
 $F \rightarrow F \uparrow P|P$
 $P \rightarrow (T)|a$
文法句型 $T^*P \uparrow (T^*F)$ 的句柄是下列符号串(C).
A.(T^*F) B. T^*F
C.P D. $P \uparrow (T^*F)$
文法句型 $T^*F \uparrow (T^*F)$ 的句柄是下列符号串(B).
A.(T^*F) B. T^*F
C.P D. $P \uparrow (T^*F)$

4.4.1.LR(0)分析

形式化定义

移进/归约冲突→

➤文法

$$G = (V_N, V_T, P, S)$$

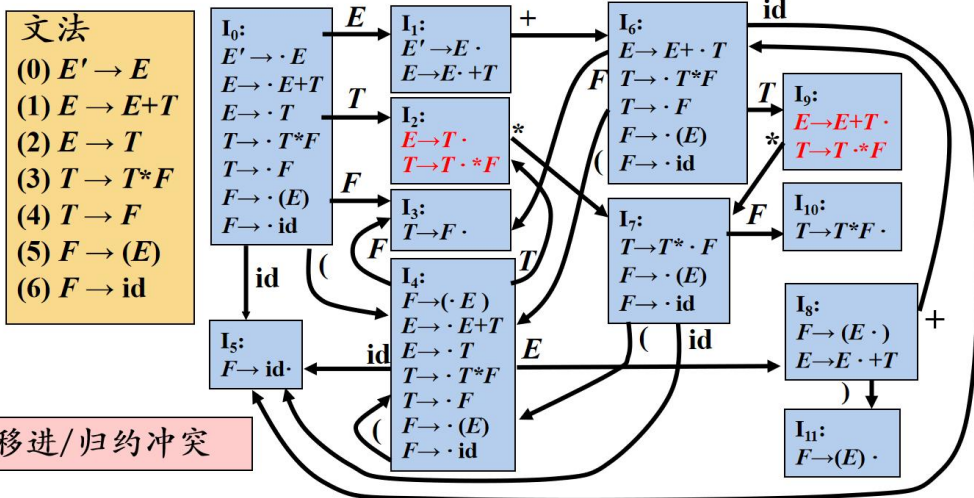
➤LR(0)自动机

$$M = (C, V_N \cup V_T, GOTO, I_0, F)$$

➤ $C = \{I_0\} \cup \{I \mid \exists J \in C, X \in V_N \cup V_T, I = GOTO(J, X)\}$

➤ $I_0 = CLOSURE(\{S' \rightarrow S\})$

➤ $F = \{CLOSURE(\{S' \rightarrow S\})\}$



状态	ACTION						GOTO		
	id	+	*	(	)	\$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2	r2	r2	r2/s7	r2	r2	r2			
3	r4	r4	r4	r4	r4	r4			
4	s5			s4			8	2	3
5	r6	r6	r6	r6	r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9	r1	r1	r1/s7	r1	r1	r1			
10	r3	r3	r3	r3	r3	r3			
11	r5	r5	r5	r5	r5	r5			

+

如果LR(0)分析表中没有语法分析动作冲突，该文法就称为LR(0)文法
不是所有CFG都能用LR(0)方法进行分析，CFG不总是LR(0)文法

LR分析器的核心部分是一张分析表，该表由(D)组成。

- A.ACTION表 B.GOTO表
C.预测分析表 D.ACTION表和GOTO表

LR分析表中的转移表(goto)是以(B)作为列标题的。

- A.终结符 B.非终结符
C.终结符或非终结符 D.表示状态的整型数

LR分析表中的动作表(action)是以(D)作为列标题的。

- A.终结符 B.非终结符
C.终结符或非终结符 D.终结符和结束符\$

X	FOLLOW(X)
E	), +, \$
T	), +, \$, *
F	), +, \$, *

||

4.4.2.SLR分析

LR(0)考虑了A的上文（规范句型的前缀），但未考虑A的下文，因此消解冲突能力有限

主要思想：设有m个移进项目 $A_m \rightarrow \alpha_m \cdot a_m \beta_m$ ，n个归约项目 $B_n \rightarrow \gamma_n \cdot$ ；

如果集合 $\{a_1, a_2, \dots, a_m\}$ 和 $FOLLOW(B_1)$ ， $FOLLOW(B_2)$ ， $\dots$ ， $FOLLOW(B_n)$ 两两不相交

则可以**根据下一个输入符号决定移进or归约or报错**

SLR分析表构造算法：

仅需将LR(0)中的“if $A \neq S'$ for $\forall a \in V_T \cup \{\$ \}$ do ACTION[i, a] = r_j (j是产生式 $A \rightarrow \alpha$ 的编号)”

替换为“if $A \neq S'$ for $\forall a \in FOLLOW(A)$ do ACTION[i, a] = r_j (j是产生式 $A \rightarrow \alpha$ 的编号)”

如果给定文法的SLR分析表中不存在有冲突的动作，那么该文法称为**SLR文法**

若状态k含有项目“ $A \rightarrow \alpha \cdot$ ”，且仅当输入符号 $a \in FOLLOW(A)$ 时，才用规则“ $A \rightarrow \alpha$ ”归约的语法分析方法是(D)。

- A.LALR分析法 B.R(0)分析法
C.LR(1)分析法 D.SLR(1)分析法

若状态k含有项目“ $A \rightarrow \alpha \cdot$ ”，对任意非终结符a，都用规则“ $A \rightarrow \alpha$ ”归约的语法分析方法是(B)。

- A.LALR分析法 B.LR(0)分析法
C.LR(1)分析法 D.SLR(1)分析法

状态	ACTION						GOTO		
	id	+	*	(	)	\$	E	T	F
0	s5			s4			1	2	3
1		s6				acc			
2		r2	s7		r2	r2			
3		r4	r4		r4	r4			
4	s5			s4			8	2	3
5		r6	r6		r6	r6			
6	s5			s4				9	3
7	s5			s4					10
8		s6			s11				
9		r1	s7		r1	r1			
10		r3	r3		r3	r3			
11		r5	r5		r5	r5			

4.4.3.LR(1)分析

SLR缺点：SLR只是简单地考察下一个输入符号b是否属于与归约项目 $A \rightarrow \alpha$ 相关联的 FOLLOW(A)，但 $b \in \text{FOLLOW}(A)$ 只是归约 α 的一个必要条件，而非充分条件。

产生式 $A \rightarrow \alpha$ 的归约，在不同使用位置，A会要求不同的后继符号。

在特定位置，A的后继符集合是FOLLOW(A)的**子集**

规范LR(1)项目： $[A \rightarrow \alpha \cdot \beta, a]$ ，其中a是一个**终结符**称为**展望符**

$[A \rightarrow \alpha \cdot \beta, a]$ 且 $\beta \neq \epsilon$ 的项中，展望符a**没有任何作用**

$[A \rightarrow \alpha \cdot, a]$ 的项在**只有在下一个输入符号等于a**时才可以按照 $A \rightarrow \alpha$ 进行归约

等价LR(1)项目： $[A \rightarrow \alpha \cdot B\beta, a]$ 在有 $B \rightarrow \gamma \in P$ 的时候 $[B \rightarrow \cdot \gamma, b]$ 等价，其中 $b \in \text{FIRST}(\beta a)$

当 $\beta \Rightarrow^+ \epsilon$ 时，此时 $b=a$ 叫继承的后继符，否则叫自生的后继符

LR(1)项目集闭包： $\text{CLOSURE}(I) = I \cup \{ [B \rightarrow \cdot \gamma, b] \mid [A \rightarrow \alpha \cdot B\beta, a] \in \text{CLOSURE}(I), B \rightarrow \gamma \in P, b \in \text{FIRST}(\beta a) \}$

GOTO 函数： $\text{GOTO}(I, X) = \text{CLOSURE}(\{ [A \rightarrow \alpha X \cdot \beta, a] \mid [A \rightarrow \alpha \cdot X\beta, a] \in I \})$

LR(1)分析表**构造算法**：

如果LR(1)分析表中没有语法分析动作冲突
那么给定的文法就称为**LR(1)文法**

➤ 文法

$$G = (V_N, V_T, P, S)$$

➤ LR(1)自动机

$$M = (C, V_N \cup V_T, \text{GOTO}, I_0, F)$$

➤ $C = \{ I_0 \} \cup \{ I \mid \exists J \in C, X \in V_N \cup V_T, I = \text{GOTO}(J, X) \}$

➤ $I_0 = \text{CLOSURE}(\{ S' \rightarrow \cdot S, \$ \})$

➤ $F = \{ \text{CLOSURE}(\{ S' \rightarrow S \cdot, \$ \}) \}$

0) $S' \rightarrow S$

1) $S \rightarrow L=R$

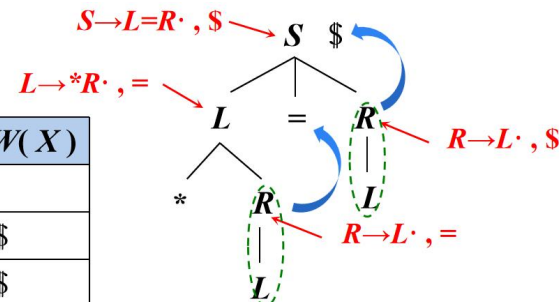
2) $S \rightarrow R$

3) $L \rightarrow *R$

4) $L \rightarrow \text{id}$

5) $R \rightarrow L$

X	FOLLOW(X)
S	\$
L	=, \$
R	=, \$



➤ 构造 G' 的规范**LR(1)**项集族 $C = \{ I_0, I_1, \dots, I_n \}$

➤ 根据 I_i 构造得到状态 i 。状态 i 的语法分析动作按照下面的方法决定：

➤ **if** $[A \rightarrow \alpha \cdot a\beta, b] \in I_i$ and $\text{GOTO}(I_i, a) = I_j$ **then** $\text{ACTION}[i, a] = sj$

➤ **if** $[A \rightarrow \alpha \cdot B\beta, b] \in I_i$ and $\text{GOTO}(I_i, B) = I_j$ **then** $\text{GOTO}[i, B] = j$

➤ **if** $[A \rightarrow \alpha \cdot, a] \in I_i$ 且 $A \neq S'$ **then** $\text{ACTION}[i, a] = rj$

➤ **if** $[S' \rightarrow S \cdot, \$] \in I_i$ **then** $\text{ACTION}[i, \$] = acc$;

➤ 没有定义的所有条目都设置为“error”

4.4.3.LR(1)分析

各种LR分析表构造方法的不同之处在于归约项目的处理上

if $[A \rightarrow a \cdot, c] \in I_i$

$\begin{cases} A = S' & ACTION[i, \$] = acc \\ A \neq S' & \begin{cases} LR(0) \text{ for } \forall a \in V_T \cup \{\$ \} & do ACTION[i, a] = r_j \text{ (j是产生式 } A \rightarrow a \text{ 的编号)} \\ SLR(1) \text{ for } \forall a \in FOLLOW(A) & do ACTION[i, a] = r_j \\ LR(1) \text{ 精准归约} & ACTION[i, c] = r_j \end{cases} \end{cases}$

4.4.4 LALR分析

如果除展望符外，两个LR(1)项目集是相同的，则称这两个LR(1)项目集是**同心的**。寻找具有**相同核心**的LR (1) 项集，并将这些项集**合并**为一个项集。

然后根据合并后得到的项集族构造语法分析表

合并同心项集**不会产生移进-归约冲突**，但是**可能产生归约-归约冲突**，也可能**推迟错误的发现**

$I_6:$
 $A \rightarrow c \cdot, d$
 $B \rightarrow c \cdot, e$

$I_9:$
 $B \rightarrow c \cdot, d$
 $A \rightarrow c \cdot, e$

当 I_6 、 I_9 合并的时候，ACTION的d列和e列就会出现矛盾，这就是**归约-归约冲突**

$I_5:$
 $A \rightarrow d \cdot, b$

$I_4:$
 $A \rightarrow d \cdot, c$

当 I_5 、 I_4 合并的之后。可能本来走到 I_5 的时候下一个输入符是c会报错，合并后没有报错而是返回了，这样到了后面会发现需要读取c的时候没有路径走而报错，这就是**错误被推迟发现**

如果分析表中没有语法分析动作冲突，给定的文法就称为LALR (1) 文法。LALR**形式**上与LR(1)相同、**大小**上与LR(0)/SLR相当

分析能力介于SLR和LR(1)二者之间 $LR(0) < SLR < LALR(1) < LR(1)$

合并后的展望符集合仍为FOLLOW集的子集

同心集合并可能会产生新的(D)冲突。

- A.二义
- B.移进/移进
- C.移进/归约
- D.归约/归约

LR(1)文法都是(C)。

- A.无二义性且无左递归
- B.可能有二义性但无左递归
- C.无二义性但可能是左递归
- D.可以既有二义性又有左递归

在LR(0)的Action表中，如果某行中存在标记为“ r_j ”的栏，则(A)。

- A.该行必定填满“ r_j ”
- B.该行未必填满“ r_j ”
- C.其他行也有“ r_j ”
- D.goto表中也有“ r_j ”

在SLR(1)的Action表中，如果某行中存在标记为“ r_j ”的栏，则(B)。

- A.该行必定填满“ r_j ”
- B.该行未必填满“ r_j ”
- C.其他行也有“ r_j ”
- D.goto表中也有“ r_j ”

一个LR(1)文法合并同心集后若不是LALR(1)文法(B)

- A.则可能存在移进/归约冲突
- B.则可能存在归约/归约冲突
- C.则可能存在移进/归约冲突和归约/归约冲突
- D.以上说法都不对

就文法的描述能力来说，有(C)。

- A.SLR(1) $\subset$ LR(0)
- B.LR(1) $\subset$ LR(0)
- C.SLR(1) $\subset$ LR(1)
- D.无二义文法 $\subset$ LR(1)

4.4.5 二义性文法的LR分析 每个二义性文法都不是LR的

$S \rightarrow iS \mid iSeS \mid a$

$I_4:$
 $S \rightarrow iS \cdot$
 $S \rightarrow iS \cdot eS$

$FOLLOW(S) = \{ e, \$ \}$

二义性算术表达式：用优先级和结合性解决冲突

状态	ACTION						GOTO
	id	+	*	(	)	\$	E
7		r1	s5		r1	r1	

文法

$E \rightarrow E+E$
 $E \rightarrow E * E$
 $E \rightarrow (E)$
 $E \rightarrow id$

$I_7:$
 $E \rightarrow E+E \cdot$
 $E \rightarrow E \cdot +E$
 $E \rightarrow E \cdot *E$

$I_3:$
 $E \rightarrow id \cdot$

$FOLLOW(E) = \{ +, *,), \$ \}$

4.4.5 LR分析中的错误处理

编译程序的语法分析器必须输出的信息是 (B)

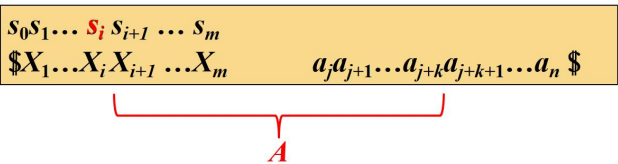
A.语法规则信息 B.语法错误信息
C.语法分析过程 D.语句序列

语法错误的检测：

当LR分析器在查询语法分析动作表时发现一个报错条目

错误恢复策略：

恐慌模式错误恢复、短语层次错误恢复



短语层次错误恢复：
手动检查设计可能的
报错方式。

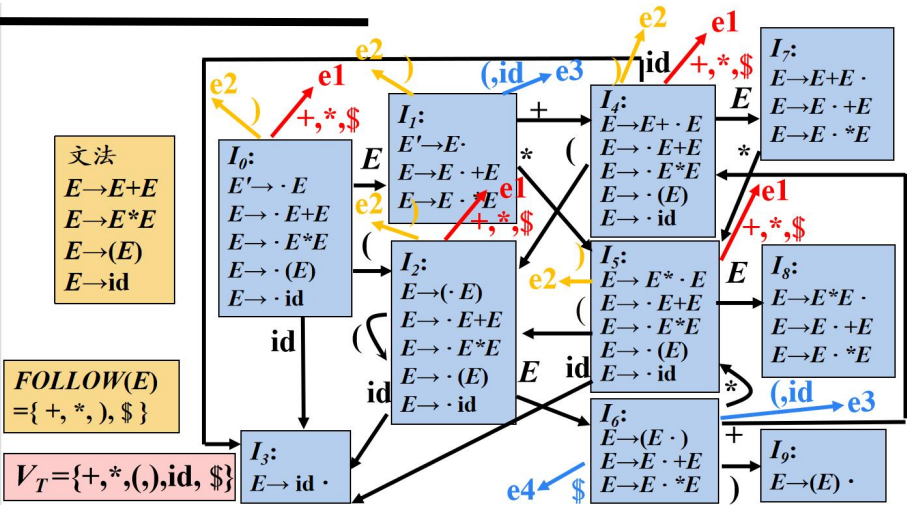
恐慌模式错误恢复：

从栈顶向下扫描，直到发现某个状态si，它有一个对应于某个非终结符A的GOTO目标，可以认为从这个A推导出的串中包含错误
然后丢弃0个或多个输入符号，直到发现一个可能合法地紧跟在A之后的符号a为止
之后将si+1 = GOTO(si, A)压入栈中，继续进行正常的语法分析

二义性if 语句文法：

选择移进else，以便让它与前面的if配对

状态	ACTION				GOTO
	i	e	a	\$	S
4		s5		r1	



状态	ACTION						GOTO
	id	+	*	(	)	\$	E
0	s3	e1	e1	s2	e2	e1	1
1	e3	s4	s5	e3	e2	acc	
2	s3	e1	e1	s2	e2	e1	6
3	r4	r4	r4	r4	r4	r4	
4	s3	e1	e1	s2	e2	e1	7
5	s3	e1	e1	s2	e2	e1	8
6	e3	s4	s5	e3	s9	e4	
7	r1	r1	s5	r1	r1	r1	
8	r2	r2	r2	r2	r2	r2	
9	r3	r3	r3	r3	r3	r3	

错误处理例程

- > e1: 将状态3压入栈中，发出诊断信息“缺少运算分量”
- > e2: 从输入中删除“)”
- > e3: 将状态4压入栈中，发出诊断信息“缺少运算符”
- > e4: 将状态9压入栈中，发出诊断信息“缺少右括号”

5.语法制导翻译

使用(A)可以定义一个程序的意义。
A.语义规则B.词法规则C.产生规则D.词法规则

一个语义动作在产生式中的位置决定了这个动作的执行时间 (✓)

语法制导翻译使用CFG来引导对语言的翻译，是一种**面向文法**的翻译技术

实现过程：对于给定的输入串x，构建x的语法分析树，并利用与产生式（**语法规则**）相关联的**语义规则**来计算分析树中各结点对应的**语义属性值**

产生式	语义规则
$D \rightarrow TL$	$L.inh = T.type$
$T \rightarrow int$	$T.type = int$
$T \rightarrow real$	$T.type = real$
$L \rightarrow L_1, id$	$L_1.inh = L.inh$
...	...

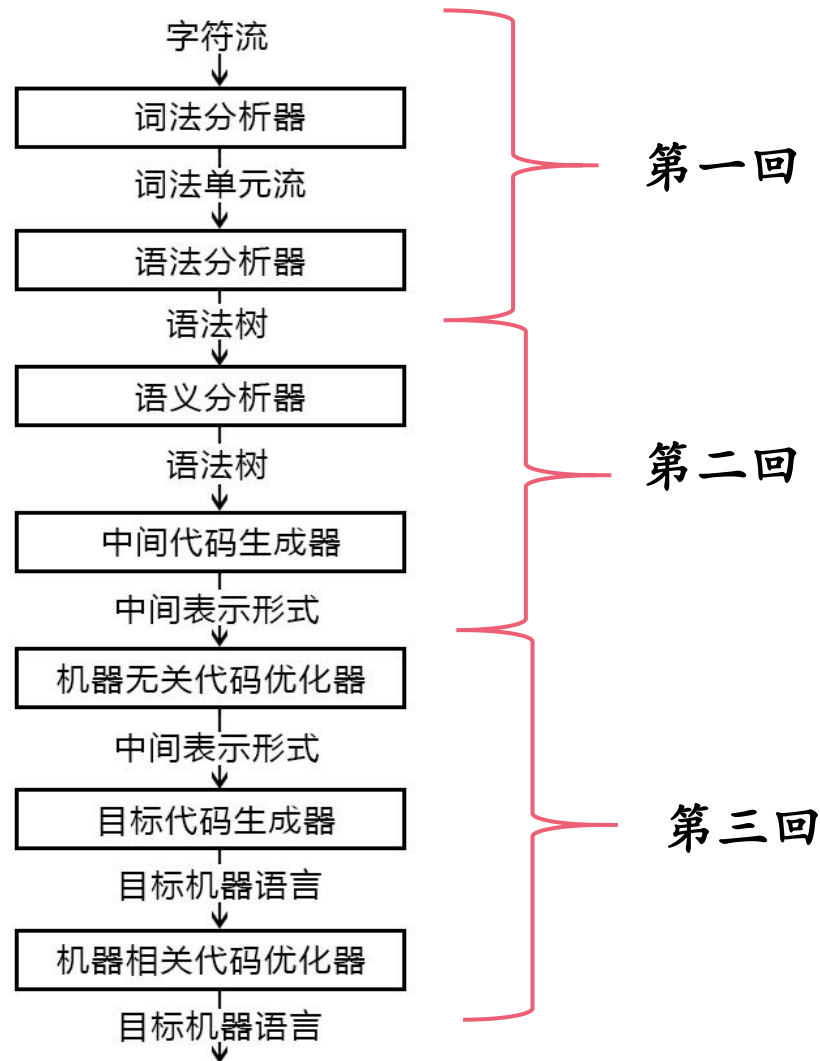
$D \rightarrow T \{ L.inh = T.type \} L$
 $T \rightarrow int \{ T.type = int \}$
 $T \rightarrow real \{ T.type = real \}$
 $L \rightarrow \{ L_1.inh = L.inh \} L_1, id$
...

语法制导翻译

CFG

语义翻译

编译系统的结构



语法制导定义(Syntax-Directed Definitions, **SDD**)

语法制导翻译方案 (Syntax-Directed Translation Scheme, **SDT**)

SDD是关于语言翻译的**高层次规格说明**，SDT是SDD的**具体实施方案**。

文法G[S]及其语法制导翻译定义如下：

产生式 语义动作
 $S' \rightarrow S$ print(S.num)
 $S \rightarrow (L)$ S.num = L.num + 1
 $S \rightarrow a$ S.num = 0
 $L \rightarrow L_1, S$ L.num = L1.num + S.num
 $L \rightarrow S$ L.num = S.num

若输入为(a,(a))，且采用自底向上的分析方法，则输出为(C)。

A.0 B.1 C.2 D.4

有文法G及其语法制导翻译如下所示
(语义规则中的*和+分别是常规意义下的算术运算符)：

$E \rightarrow E(1) \wedge T \{ E.val = E(1).val * T.val \}$
 $E \rightarrow T \{ E.val = T.val \}$
 $T \rightarrow T(1) \# n \{ T.val = T(1).val + n.val \}$
 $T \rightarrow n \{ T.val = n.val \}$
则分析句子 $3 \wedge 3 \# 4$ 其值为(B)。
A.10 B.21 C.14 D.24

有一语法指导定义如下：

$S \rightarrow bAb$ print "1"
 $A \rightarrow (B$ print "2"
 $A \rightarrow a$ print "3"
 $B \rightarrow aA$ print "4"

若输入序列为b(a(a(aa)))b，且采用自底向上的分析方法，则输出序列为(B)。

A.32224441 B.34242421
C.12424243 D.34442212

5.1.语法制导翻译语法制导定义SDD

有一语法指导定义如下，其中+表示符号连接运算：

$S \rightarrow B \quad \text{print } B.\text{vers}$
 $B \rightarrow a \quad B.\text{vers} = a \qquad B \rightarrow b \quad B.\text{vers} = b$
 $B \rightarrow Ba \quad B.\text{vers} = a + B.\text{vers} \qquad B \rightarrow Bb \quad B.\text{vers} = b + B.\text{vers}$

若输入序列为abab，且采用自底向上的分析方法，则输出序列为(D)。

A.aabb B.abab C.bbba D.baba

语法制导定义SDD是对CFG的推广

- 将每个文法符号和一个语义属性集合相关联
- 将每个产生式 and 一组语义规则相关联，用来计算该产生式中各文法符号的属性值

文法符号的属性

综合属性

非终结符：通过N的子结点或N本身的属性值来定义。终结符：词法分析器提供的词法值

继承属性

非终结符：通过N的父结点、N的兄弟结点或N本身的属性值来定义。终结符：没有继承属性

属性文法

一个没有副作用的SDD有时也称为属性文法，属性文法的规则仅仅通过其它属性值和常量来定义一个属性值（副作用是在翻译过程中多余的动作，并且对语法翻译没有产生影响的动作。eg. print(E.val)）

注释分析树：每个节点都带有属性值的分析树

依赖图：一个描述了分析树中结点属性间依赖关系的有向图（如右图）

分析树中每个标号为X的结点的每个属性a都对应着依赖图中的一个结点

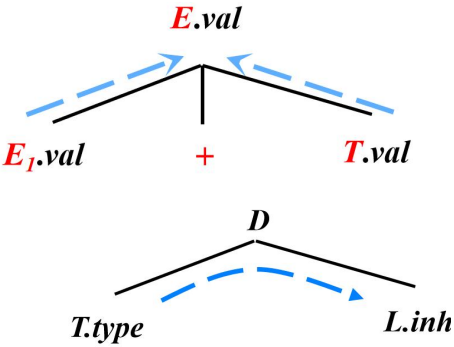
如果属性X.a的值依赖于属性Y.b的值，则依赖图中有一条从Y.b的结点指向X.a的结点的有向边

通过依赖关系计算拓扑排序得到计算顺序。给定SDD很难直接判断是否存在求值顺序，但是对SDD的某些有用子类，能够保证对每棵语法分析树都存在一个求值顺序，

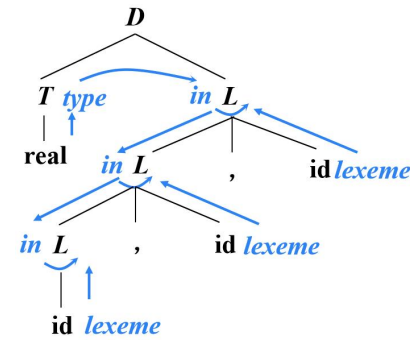
如：S-属性定义(S-SDD)、L-属性定义(L-SDD)

终结符具有(B)属性。
A.继承B. 综合
C.传递D.抽象

- 以下说法正确的是(A)。
- A.语义规则中的属性有两种：综合属性与继承属性
 - B.终结符只有继承属性，它由词法分析器提供
 - C.非终结符可以有综合属性，但不能有继承属性
 - D.属性值在分析过程中可以进行计算，但不能传递



real a,b,c



	产生式	语义规则
(1)	$D \rightarrow T L$	$L.inh = T.type$
(2)	$T \rightarrow \text{int}$	$T.type = \text{int}$
(3)	$T \rightarrow \text{real}$	$T.type = \text{real}$
(4)	$L \rightarrow L_1, \text{id}$	$L_1.inh = L.inh$ $\text{addtype}(\text{id.lexeme}, L.inh)$
(5)	$L \rightarrow \text{id}$	$\text{addtype}(\text{id.lexeme}, L.inh)$

5.2. S-属性定义和L-属性定义

每个S-属性定义都是L-属性定义 (✓)

S-属性定义: 仅仅使用**综合属性**的SDD, 又称**S属性的SDD**、**S-SDD**

S-属性定义可以在**自底向上的语法分析**过程中实现

L-属性定义: 它的每个属性要么是一个**综合属性**, 要么是满足如下条件的**继承属性**:

假设存在一个产生式 $A \rightarrow X_1 X_2 \cdots X_n$, 其右部符号 $X_i (1 \leq i \leq n)$ 的继承属性仅依赖于下列属性:

- ① A的**继承属性**
- ② 产生式中 X_i **左边**的符号 $X_1, X_2, \cdots, X_{i-1}$ 的属性
- ③ X_i 本身的属性, 但 X_i 的全部属性不能在依赖图中形成环路

综合属性 ← 子节点的属性

继承属性 ← 父节点的**继承**属性
左兄弟节点的属性

虚属性: 写成过程调用或过程段的语义规则。eg. `addtype(id.lexeme, L.inh)`

5.3. 语法制导翻译方案SDT

语法制导翻译方案(SDT)是在**产生式右部**中嵌入了**程序片段**(称为**语义动作**)的CFG

SDT可以看作是SDD的**具体实施方案**

5.3.1 将**S-SDD转换为SDT**: 将每个语义动作都放在产生式的最后

如果一个S-SDD的基本文法可以使用**LR分析技术**, 那么它的SDT可以在LR语法分析过程中实现(当归约发生时执行相应的语义动作)

为每个栈记录增加**属性值字段**, 存放文法符号的**综合属性值**

在每次**归约**时**调用**计算综合属性值的**语义子程序**

若支持多个属性

方法1: 使栈记录变得足够大

方法2: 在栈记录中存放指针

$A \rightarrow XYZ \{ A.a = f(X.x, Y.y, Z.z) \}$

将语义动作中的**抽象定义式**
改写成**具体可执行的栈操作**

value
symbol
state

	...	$X.x$	$Y.y$	$Z.z$
value				
symbol	\$	X	Y	Z
state	S_0	S_X	S_Y	S_Z

top



```
stack[top-2].symp = A
stack[top-2].val = f( stack[top-2].val, stack[top-1].val, stack[top].val )
top = top-2;
```

value
symbol
state

	...	$A.a$
value		
symbol	\$	A
state	S_0	S_A

top



关于将L-SDD转换为SDT的规则, 以下选项中, 正确的是(C)。

- A. 将计算某个非终结符号A的继承属性的动作放在产生式的最后
- B. 将计算一个产生式左部符号的继承属性的动作放置在产生式的最后
- C. 将计算某个非终结符号A的继承属性的动作插入到产生式右部中紧靠在A的本次出现之前的位置上
- D. 将每个语义动作都放在产生式的最后

以下说法不正确的是(C)。

- A. 使用语法制导翻译方案的编译程序能同时进行语法分析和语义分析
- B. 语法制导翻译方案(SDT)是在产生式右部中嵌入了程序片段(称为语义动作)的CFG
- C. SDD可以看作是SDT的具体实施方案
- D. 将一个S-SDD转换为SDT的方法是: 将每个语义动作都放在产生式的最后

5.3.语法制导翻译方案SDT

5.3.2将L-SDD转换为SDT

将L-SDD转换为SDT的规则

将计算一个产生式左部符号的**综合属性**的动作放置在这个产生式右部的**最右端**

将计算某个非终结符号A的**继承属性**的动作插入到产生式右部中**紧靠在A的本次出现之前的位置上**

如果一个L-SDD的基本文法可以使用**LL分析技术**，那么它的SDT可以在**LL或LR语法分析过程中实现**

①②在**递归/非递归的预测分析过程**中进行语义翻译③在**LR分析过程**中进行语义翻译

- 1) $T \rightarrow F \{ T'.inh = F.val \} T' \{ T.val = T'.syn \}$
2) $T' \rightarrow *F \{ T_1'.inh = T'.inh \times F.val \} T_1' \{ T'.syn = T_1'.syn \}$
3) $T' \rightarrow \varepsilon \{ T'.syn = T'.inh \}$
4) $F \rightarrow \text{digit} \{ F.val = \text{digit.lexval} \}$

以下说法不正确的是(A)。

- A. 如果一个S-SDD的基本文法可以使用LR分析技术，那么它的SDT可以在LL语法分析过程中实现
B. 如果一个S-SDD的基本文法可以使用LR分析技术，那么它的SDT可以在LR语法分析过程中实现
C. 如果一个L-SDD的基本文法可以使用LL分析技术，那么它的SDT可以在LL语法分析过程中实现
D. 如果一个L-SDD的基本文法可以使用LL分析技术，那么它的SDT可以在LR语法分析过程中实现

5.4.L-属性定义的自顶向下翻译

5.4.1.在非递归的预测分析过程中进行翻译

原先

栈	X
	Y
	Z
	\$

现在：扩展语法分析栈

T	Tsyn	S
	val	

SDT

- 1) $T \rightarrow F \{ a_1 \} T' \{ a_2 \}$
2) $T' \rightarrow *F \{ a_3 \} T_1' \{ a_4 \}$
3) $T' \rightarrow \varepsilon \{ a_5 \}$
4) $F \rightarrow \text{digit} \{ a_6 \}$

- $a_1: T'.inh = F.val$
 $a_2: T.val = T'.syn$
 $a_3: T_1'.inh = T'.inh \times F.val$
 $a_4: T'.syn = T_1'.syn$
 $a_5: T'.syn = T'.inh$
 $a_6: F.val = \text{digit.lexval}$

(1) 增加**属性值 (value)** 字段 (**下方那一行**)

(2) 对于**非终结符A**，将**继承属性**和**综合属性**存放在不同的记录中 (如T'和T'syn)

(3) **终结符**的**综合属性**存放在其记录的属性值字段

(4) 增加**动作记录**用来存放指向**语义动作代码**的指针

(5) 不光是动作记录，其实分析栈中的**每一个记录**都对应着一段**执行代码**(如下方产生式 (1) 对应的表)

①**综合记录出栈时**，要将**综合属性值**复制给后面特定的语义动作

F	Fsyn	{a ₁ }	T'	T'syn	{a ₂ }	Tsyn	S
	val		inh	syn		val	

{a ₁ }	T'	T'syn	{a ₂ }	Tsyn	S
Fval=3	inh=3	syn		val	

②**变量展开时** (即变量本身的记录**出栈**时)，如果其含有**继承属性**，则要将**继承属性值**复制给后面特定的语义动作

1) $T \rightarrow F \{ a_1: T'.inh = F.val \} T' \{ a_2: T.val = T'.syn \}$

符号	属性	执行代码
F		
Fsyn	val	stack[top-1].Fval = stack[top].val; top=top-1;
a ₁	Fval	stack[top-1].inh = stack[top].Fval; top=top-1;
T'	inh	根据当前输入符号选择产生式进行推导 若选 2): stack[top+3].T'.inh = stack[top].inh; top=top+6; 若选 3): stack[top].T'.inh = stack[top].inh;
T'syn	syn	stack[top-1].T'syn = stack[top].syn; top=top-1;
a ₂	T'syn	stack[top-1].val = stack[top].T'syn; top=top-1;

*	F	Fsyn	{a ₃ }	T ₁ '	T ₁ 'syn	{a ₄ }	T'syn	{a ₂ }	Tsyn	S
		val	T'.inh=3	inh	syn		syn		val	

5.4.2.在递归的预测分析过程中进行翻译

SDT

- 1) $T \rightarrow F \{ T'.inh = F.val \} T'$
 { $T.val = T'.syn$ }
- 2) $T' \rightarrow *F \{ T_1'.inh = T'.inh \times F.val \} T_1'$
 { $T'.syn = T_1'.syn$ }
- 3) $T' \rightarrow \varepsilon \{ T'.syn = T'.inh \}$
- 4) $F \rightarrow \text{digit} \{ F.val = \text{digit.lexval} \}$

Desent()

```
{  
    D: Tval;  
    Getnext(token);  
    Tval = T(token);  
    if token ≠ "$" then Error;  
    return ;  
}
```

非终结符A的代码根据当前的输入决定
使用哪个产生式（根据SELECT集）

在非递归的预测分析过程中进行翻译，以下说法不正确的是(D)。
A.要想在非递归的预测分析过程中进行翻译，需要扩展语法分析栈
B.非终结符A的继承属性和综合属性的计算时机不同
C.将非终结符A的继承属性和综合属性存放在不同的记录中
D.综合属性在A出现之前就可以计算

在非递归的预测分析过程中进行翻译，以下说法不正确的是(D)。
A.要想在非递归的预测分析过程中进行翻译，需要扩展语法分析栈
B.综合记录用于存放非终结符综合属性值
C.动作记录，用来存放指向将被执行的语义动作代码的指针
D.综合属性存放在A本身的记录中

在非递归的预测分析过程中进行翻译，以下说法不正确的是(D)。
A.分析栈中的每一个记录都对应着一段执行代码
B.综合记录出栈时，要将综合属性值复制给后面特定的语义动作
C.变量展开时(即变量本身的记录出栈时)，如果其含有继承属性，则要将继承属性值复制给后面特定的语义动作
D.继承属性在A的儿子们都分析完毕之后才能计算

为每个非终结符A构造一个函数，A
的每个继承属性对应该函数的一个形
参，函数的返回值是A的综合属性值

对出现在A产生式右部中的
每个文法符号的每个属
性都设置一个局部变量

对于每个动作，将其代码复制到
语法分析器，并把对属性的引用
改为对相应变量的引用

```
T'syn T'(token, T'inh)  
{ D: Fval, T1'inh, T1'syn;  
  if token="*" then  
  { Getnext(token);  
    Fval=F(token);  
    T1'inh= T'inh × Fval;  
    T1'syn=T1'(token, T1'inh);  
    T'syn=T1'syn;  
    return T'syn;  
  }  
  else if token= "$" then  
  { T'syn= T'inh;  
    return T'syn;  
  }  
  else Error;  
}
```

在递归的预测分析过程中进行翻译，以下说法不正确的是(D)。
A.可以将一个递归的预测分析器扩展为一个翻译器
B.在语法分析器中，每个非终结符A对应一个过程，在做语义分析时，要将过程扩展成一个函数
C.以继承属性作为函数的参数，以综合属性作为函数的返回值
D.以综合属性作为函数的参数，以继承属性作为函数的返回值

在递归的预测分析过程中进行翻译，以下说法不正确的是(C)。
A.在语法分析器中，每个非终结符A对应一个过程，在做语义分析时，要将过程扩展成一个函数
B.对出现在A产生式右部中的每个文法符号的每个属性都设置一个局部变量
C.如果非终结符含有继承属性，需要将函数调用的返回值赋给相应的局部变量
D.对于产生式右部的每个动作，将其代码复制到语法分析器，并把对属性的引用改为对相应变量的引用

以下说法不正确的是(A)。
A.语法制导翻译方案只限自底向上的分析方法
B.给定一个以LL文法为基础的L-SDD，可以修改这个文法，并在LR语法分析过程中计算这个新文法之上的SDD
C.对于这个内嵌的语义动作，向文法中引入一个标记非终结符M来替换它
D.每个标记非终结符M对应着一个空产生式 $M \rightarrow \varepsilon$ ，该产生式对应着一段语义子程序，它的任务就是完成M所替换的那个语义动作要完成的工作

5.5.L-属性定义的自底向上翻译

给定一个以LL文法为基础的L-SDD，可以修改这个文法，使得所有语义动作都位于产生式末尾（类似S-SDD使用，可以理解为将现有的继承属性变为引入的非终结符的综合属性）

修改方法：对每个内嵌的语义动作，向文法中引入一个标记非终结符来替换它。每个这样的位置都有一个不同的标记，并且对于任意一个标记M都有一个产生式 $M \rightarrow \epsilon$
如果标记非终结符M在某个产生式 $A \rightarrow \alpha \{a\} \beta$ 中替换了语义动作a，对a进行修改得到a'，并且将a'关联到 $M \rightarrow \epsilon$ 上。动作a'

- (a) 将动作a需要的A或α中符号的任何属性作为M的继承属性进行复制
- (b) 按照a中的方法计算各个属性，但是将计算得到的这些属性作为M的综合属性（作为下一个符号的继承属性）

给定一个以LL文法为基础的L-属性定义，可以修改这个文法，并在LR语法分析过程中计算这个新文法之上的SDD。（√）

在各个非终结符之前放置语义动作来计算它的继承属性，并在产生式后端放置语义动作计算综合属性。（√）

在各个非终结符之前放置语义动作来计算它的综合属性，并在产生式后端放置语义动作计算继承属性。（×）

对每个内嵌的语义动作，向文法中引入一个标记非终结符来替换它。每个这样的位置都有一个不同的标记，并且对于任意一个标记M都有一个产生式 $M \rightarrow \epsilon$ 。（√）

1) $T \rightarrow F M T' \{ T.val = T'.syn \}$
 $M \rightarrow \epsilon \{ M.i = F.val; M.s = M.i \}$
2) $T' \rightarrow * F N T_1' \{ T'.syn = T_1'.syn \}$
 $N \rightarrow \epsilon \{ N.i1 = T'.inh;$
 $N.i2 = F.val;$
 $N.s = N.i1 \times N.i2 \}$
3) $T' \rightarrow \epsilon \{ T'.syn = T'.inh \}$
4) $F \rightarrow digit \{ F.val = digit.lexval \}$

- 1) $T \rightarrow F M T' \{ stack[top-2].val = stack[top].syn; top = top-2; \}$
 $M \rightarrow \epsilon \{ stack[top+1].T.inh = stack[top].val; top = top+1; \}$
- 2) $T' \rightarrow * F N T_1' \{ stack[top-3].syn = stack[top].syn; top = top-3; \}$
 $N \rightarrow \epsilon \{ stack[top+1].T.inh = stack[top-2].T.inh \times stack[top].val; top = top+1; \}$
- 3) $T' \rightarrow \epsilon \{ stack[top+1].syn = stack[top].T.inh; top = top+1; \}$
- 4) $F \rightarrow digit \{ stack[top].val = stack[top].lexval; \}$

value		...	F.val	T'.inh	T'.syn
symbol	\$	...	F	M	T'
state	S ₀	...	S _F	S _M	S _T

6.中间代码生成

6.1.声明语句翻译

```
struct stype
{ char[8] name;
  int score;
};
stype[50] table;
stype* p;
```

和stype绑定的类型表达式: record((name×array(8,char))×(score×integer))
和table绑定的类型表达式: array(50,stype)
和p绑定的类型表达式: pointer(stype)

用(C)可以把a:=b+c翻译成四元式序列。
A.语法规则 B.词法规则
C.语义规则 D.等价变换规则

声明语句翻译的主要任务: 收集标识符的**类型**等属性信息, 并为每一个名字分配一个**相对地址**
名字的**类型**和**相对地址**信息保存在相应的**符号表**记录中

- ①**基本类型**是类型表达式; ②可以为类型表达式命名, **类型名**也是类型表达式
- ③将**类型构造符**作用于**类型表达式**可以构成新的类型表达式

在编译阶段只对可执行语句进行翻译。(×)
在程序中标识符的出现仅为定义性的。(×)
在程序中标识符的出现仅为使用性的。(×)

数组构造符: **array** (*I*, *T*) 指针构造符: **pointer** (*T*) 笛卡尔乘积构造符: $T_1 \times T_2$
函数构造符: $T_1 \times T_2 \times \dots \times T_n \rightarrow R$ 记录构造符: **record** (($N_1 \times T_1$) × ($N_2 \times T_2$) × ... × ($N_n \times T_n$))

目的: 从类型表达式可以知道该类型在**运行时刻**所需的存储单元数量称为**类型的宽度**; 在编译时刻, 可以使用**类型的宽度**为每一个名字分配一个**相对地址**

enter(name, type, offset):在符号表中为名字name创建记录,将name的类型设置为type,相对地址设置为offset

- 中间代码生成时所依据的是(C)。
A.语法规则 B.词法规则
C.语义规则 D.等价变换规则
- 在编译程序中与中间代码生成无关的是(D)。
A.便于目标代码的优化 B.便于存储空间的组织
C.便于编译程序的移植 D.便于目标代码的移植

- 以下说法不正确的是(D)。
- A.对于声明语句, 语义分析的主要任务就是收集标识符的类型等属性信息, 为每一个名字分配一个相对地址
 - B.从变量类型可以知道该变量在运行时刻需要的内存数量。在编译时刻, 可以使用这些数量为每一个名字分配一个相对地址
 - C.名字的类型和相对地址信息保存在相应的符号表条目中
 - D.对声明的处理要构造符号表, 但不产生中间代码

- 以下说法不正确的是(B)。
- A.类型自身也有结构, 用类型表达式来表示这种结构
 - B.基本类型不是类型表达式
 - C.类型名也是类型表达式
 - D.将类型构造符作用于类型表达式可以构成新的类型表达式

- ① $P \rightarrow \{ \text{offset} = 0 \} D$
- ② $D \rightarrow T \text{ id}; \{ \text{enter}(\text{id.lexeme}, T.\text{type}, \text{offset}); \text{offset} = \text{offset} + T.\text{width}; \} D$
- ③ $D \rightarrow \varepsilon$
- ④ $T \rightarrow B \quad \{ t = B.\text{type}; w = B.\text{width}; \}$
 $\quad C \quad \{ T.\text{type} = C.\text{type}; T.\text{width} = C.\text{width}; \}$
- ⑤ $T \rightarrow \uparrow T_1 \{ T.\text{type} = \text{pointer}(T_1.\text{type}); T.\text{width} = 4; \}$
- ⑥ $B \rightarrow \text{int} \{ B.\text{type} = \text{int}; B.\text{width} = 4; \}$
- ⑦ $B \rightarrow \text{real} \{ B.\text{type} = \text{real}; B.\text{width} = 8; \}$
- ⑧ $C \rightarrow \varepsilon \quad \{ C.\text{type} = t; C.\text{width} = w; \}$
- ⑨ $C \rightarrow [\text{num}] C_1 \{ C.\text{type} = \text{array}(\text{num.val}, C_1.\text{type}); C.\text{width} = \text{num.val} * C_1.\text{width}; \}$

符号	综合属性
<i>B</i>	<i>type, width</i>
<i>C</i>	<i>type, width</i>
<i>T</i>	<i>type, width</i>

6.2.赋值语句的翻译

6.2.1简单赋值语句的翻译

赋值语句翻译的主要任务：生成对表达式求值的三地址码
newtemp()：生成一个新的临时变量t，返回t的地址
gen(code)：生成三地址指令code
lookup(name)：查询符号表返回name对应的记录

符号	综合属性
<i>S</i>	<i>code</i>
<i>E</i>	<i>code</i> <i>addr</i>

- ① $S \rightarrow id = E; \{ p = lookup(id.lexeme);$
 $\text{if } p == nil \text{ then error};$
 $gen(p \text{ '=' } E.addr); \}$
- ② $E \rightarrow E_1 + E_2 \{ E.addr = newtemp();$
 $gen(E.addr \text{ '=' } E_1.addr \text{ '+' } E_2.addr); \}$
- ③ $E \rightarrow E_1 * E_2 \{ E.addr = newtemp();$
 $gen(E.addr \text{ '=' } E_1.addr \text{ '*' } E_2.addr); \}$
- ④ $E \rightarrow -E_1 \{ E.addr = newtemp();$
 $gen(E.addr \text{ '=' } 'uminus' E_1.addr); \}$
- ⑤ $E \rightarrow (E_1) \{ E.addr = E_1.addr; \}$
- ⑥ $E \rightarrow id \{ E.addr = lookup(id.lexeme);$
 $\text{if } E.addr == nil \text{ then error}; \}$

$S \rightarrow id = E; \{ p = lookup(id.lexeme); \text{if } p == nil \text{ then error};$
 $S.code = E.code \parallel$
 $gen(p \text{ '=' } E.addr); \}$
 $E \rightarrow E_1 + E_2 \{ E.addr = newtemp();$
 $E.code = E_1.code \parallel E_2.code \parallel$
 $gen(E.addr \text{ '=' } E_1.addr \text{ '+' } E_2.addr); \}$
 $E \rightarrow E_1 * E_2 \{ E.addr = newtemp();$
 $E.code = E_1.code \parallel E_2.code \parallel$
 $gen(E.addr \text{ '=' } E_1.addr \text{ '*' } E_2.addr); \}$
 $E \rightarrow -E_1 \{ E.addr = newtemp();$
 $E.code = E_1.code \parallel$
 $gen(E.addr \text{ '=' } 'uminus' E_1.addr); \}$
 $E \rightarrow (E_1) \{ E.addr = E_1.addr;$
 $E.code = E_1.code; \}$
 $E \rightarrow id \{ E.addr = lookup(id.lexeme); \text{if } E.addr == nil \text{ then error};$
 $E.code = ' '; \}$

eg. $x = (a + b) * c; \rightarrow t_1 = a + b$
 $t_2 = t_1 * c$
 $x = t_2$

6.2.2数组引用的翻译

将数组译成三地址码时要解决的主要问题是确定数组元素的存放地址，也就是数组元素的寻址

$a[i_1][i_2] \dots [i_k]: base + i_1 \times w_1 + i_2 \times w_2 + \dots + i_k \times w_k$
 $w_k \rightarrow a[i_1][i_2] \dots [i_k]$ 的宽度

$eg.type(a) = array(3, array(5, array(8, int)))$
 $addr(a[i_1][i_2][i_3])$

$= base + i_1 * w_1 + i_2 * w_2 + i_3 * w_3$
 $= base + i_1 * 160 + i_2 * 32 + i_3 * 4$

- 以下说法不正确的是(D)。
- A.赋值语句翻译的主要任务是生成对表达式求值的三地址码
 - B.在增量翻译方法中，gen()函数不仅要构造出一个新的三地址指令，还要将它添加到至今为止已生成的指令序列之后
 - C.如果一个赋值语句中涉及到数组元素，那么将该语句翻译成三地址码时要解决的主要问题是确定数组元素的存放地址，也就是数组元素的寻址
 - D.数组元素的地址计算与数组的存储方式无关

`int abc;`
`int i;`
`int myarray[3][4]`

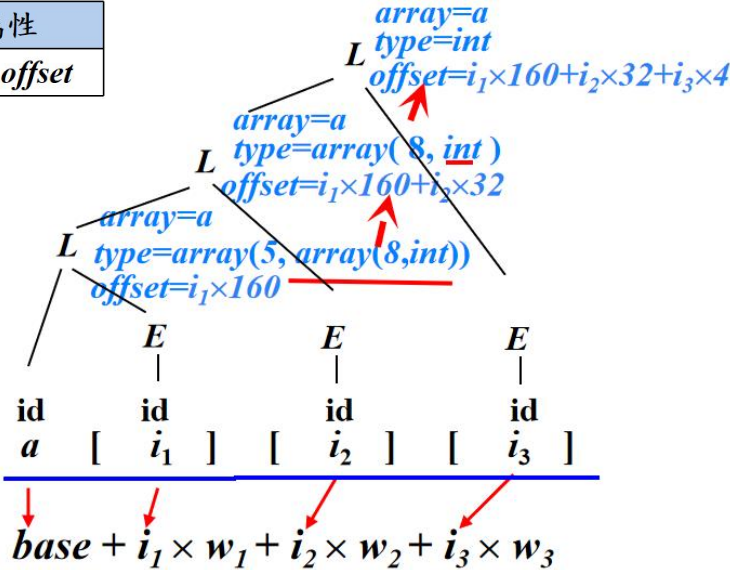
名字	基本属性				扩展属性								
	种属	类型	地址	扩展属性指针									
abc	变量	int	0	NULL	维数 各维维长 <table><tr><td rowspan="2">2</td><td>3</td><td>4</td></tr><tr><td>16</td><td>4</td></tr><tr><td colspan="3">各维宽度</td></tr></table>	2	3	4	16	4	各维宽度		
2	3	4											
	16	4											
各维宽度													
i	变量	int	4	NULL									
myarray	数组	int	8										
...	...												

赋值语句基本文法
 $S \rightarrow id = E; \mid L = E;$
 $E \rightarrow E1 + E2 \mid -E1 \mid (E1) \mid id \mid L$
 $L \rightarrow id [E] \mid L1 [E]$

设置type属性：计算宽度w
设置offset属性：累积公式中的偏移地址
设置array属性：传递数组的基地址

三地址码
 $t_1 = i_1 * 160$
 $t_2 = i_2 * 32$
 $t_3 = t_1 + t_2$
 $t_4 = i_3 * 4$
 $t_5 = t_3 + t_4$
 $t_6 = a[t_5]$

符号	综合属性
<i>L</i>	<i>array, type, offset</i>



6.3.控制语句的翻译

控制流语句的基本文法

$P \rightarrow S$ $S \rightarrow S_1 S_2$ $S \rightarrow \text{id} = E ; | L = E ;$

$S \rightarrow \text{if } B \text{ then } S_1 | \text{if } B \text{ then } S_1 \text{ else } S_2 | \text{while } B \text{ do } S_1$

newlabel(): 生成一个用于存放标号的新的临时变量L, 返回变量地址

label(L): 将下一条三地址指令的标号存放地址L中

$P \rightarrow \{ S.next = \text{newlabel}(); \} S \{ \text{label}(S.next); \}$

$S \rightarrow \{ S_1.next = \text{newlabel}(); \} S_1 \{ \text{label}(S_1.next); S_2.next = S.next; \} S_2$

$S \rightarrow \text{if } \{ B.true = \text{newlabel}(); B.false = \text{newlabel}(); \} B \text{ then } \{ S_1.next = S.next; \text{label}(B.true); \} S_1 \{ \text{gen('goto'}$

$S.next) \} \text{else } \{ S_2.next = S.next; \text{label}(B.false); \} S_2$

$S \rightarrow \text{if } \{ B.true = \text{newlabel}(); B.false = S.next; \} B \text{ then } \{ S_1.next = S.next; \text{label}(B.true); \} S_1$

$S \rightarrow \text{while } \{ B.true = \text{newlabel}(); B.false = S.next; B.begin = \text{newlabel}(); \text{label}(B.begin); \} B \text{ do } \{ S_1.next =$

$B.begin; \text{label}(B.true); \} S_1 \{ \text{gen('goto' } B.begin); \}$

SDT编写要点: ①先计算继承属性; ②观察是否有导入箭头。如果有, 调用label()函数

③上一个代码框执行完不顺序执行下一个代码框时, 生成一条显式跳转指令

④有自下而上的箭头时, 设置begin属性。且定义后直接调用label()函数绑定地址

布尔表达式的翻译

$B \rightarrow B \text{ or } B | B \text{ and } B | \text{not } B | (B) | E \text{ relop } E | \text{true} | \text{false}$

在跳转代码中, 逻辑运算符&&、|| 和!被翻译成跳转指令。运算符本身不出现在代码中,

布尔表达式的值是通过代码序列中的位置来表示的

$B \rightarrow E_1 \text{ relop } E_2 \{ \text{gen('if' } E_1.addr \text{ relop } E_2.addr \text{ 'goto' } B.true); \text{gen('goto' } B.false); \}$

$B \rightarrow \text{true} \{ \text{gen('goto' } B.true); \}$

$B \rightarrow \text{false} \{ \text{gen('goto' } B.false); \}$

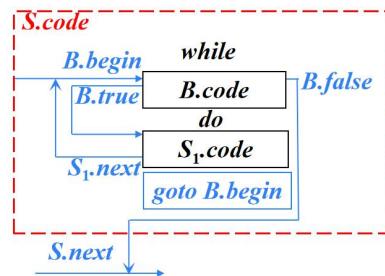
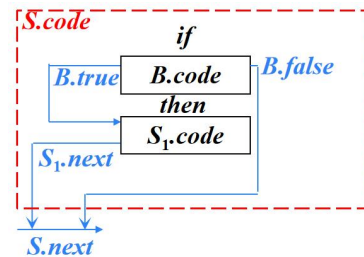
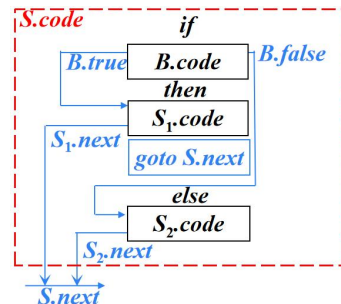
$B \rightarrow (\{ B_1.true = B.true; B_1.false = B.false; \} B_1)$

$B \rightarrow \text{not} \{ B_1.true = B.false; B_1.false = B.true; \} B_1$

$B \rightarrow \{ B_1.true = B.true; B_1.false = \text{newlabel}(); \} B_1 \text{ or } \{ B_2.true = B.true; B_2.false = B.false; \text{label}(B_1.false); \} B_2$

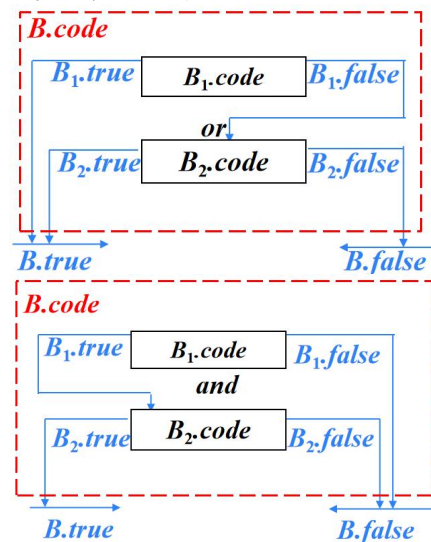
$B \rightarrow \{ B_1.true = \text{newlabel}(); B_1.false = B.false; \} B_1 \text{ and } \{ B_2.true = B.true; B_2.false = B.false; \text{label}(B_1.true); \} B_2$

符号	继承属性
S	next
B	true,false,begin



关于布尔表达式的叙述, 不正确的是(B)。

- A. 布尔常量是布尔表达式
- B. 布尔常量不是布尔表达式
- C. 关系表达式是布尔表达式
- D. 将括号和逻辑运算符作用于布尔表达式得到一个新的布尔表达式



下列SDT中不正确的是(CE)。

- A. $B \rightarrow \text{true} \{ \text{gen('goto' } B.true); \}$
- B. $B \rightarrow \text{false} \{ \text{gen('goto' } B.false); \}$
- C. $B \rightarrow \text{not} \{ B_1.true = B.true; B_1.false = B.false; \} B_1$
- D. $B \rightarrow (\{ B_1.true = B.true; B_1.false = B.false; \} B_1)$
- E. $B \rightarrow (B_1 \{ B_1.true = B.true; B_1.false = B.false; \})$

6.3.控制语句的翻译

SDT的通用实现方法

首先建立一棵语法分析树

然后按照从左到右的**深度优先**顺序来执行这些动作

SDD: L-SDD

赋值语句: 只定义了**综合属性**

分支、循环语句: 只定义了**继承属性**, 且不依赖右兄弟节点属性值

基础文法: 可以使用**LR分析技术**

在分支和循环中会用到条件式, 而用作条件式的通常是布尔表达式。(✓)

在控制流语句的翻译中, 布尔表达式B被翻译成由跳转指令构成的跳转代码。(✓)

逻辑运算符&&、|| 和 ! 会出现在代码中。(✗)

四元式之间的联系是通过(B)实现的。

A.指示器 B.临时变量

C.符号表 D.程序变量

四元式表示法的优点为(C)。

A.不便于优化处理, 但便于表的更动

B.不便于优化处理, 但节省存储空间

C.便于优化处理, 也便于表的更动

D.便于表的更动, 也节省存储空间

为了进行左边语句所示的优化, 进行如下修改:

(**fall**就是不执行任何操作, 即不跳转, 顺序执行)

• 修改if-then语句的SDT

$S \rightarrow \text{if } \{ B.\text{true} = \text{fall}; B.\text{false} = S.\text{next}; \} B \text{ then } \{ S_1.\text{next} = S.\text{next}; \} S_1$

• 修改 $B \rightarrow E_1 \text{ relop } E_2$ 的SDT

$B \rightarrow E_1 \text{ relop } E_2 \{$

$\text{if } B.\text{true} \neq \text{fall and } B.\text{false} \neq \text{fall then gen('if' } E_1.\text{addr relop } E_2.\text{addr 'goto' } B.\text{true}); \text{ gen('goto' } B.\text{false}); \}$

$\text{else if } B.\text{true} \neq \text{fall then gen('if' } E_1.\text{addr relop } E_2.\text{addr 'goto' } B.\text{true});$

$\text{else if } B.\text{false} \neq \text{fall then gen('ifFalse' } E_1.\text{addr relop } E_2.\text{addr 'goto' } B.\text{false}); \text{ else ' ' } \}$

• 修改 $B \rightarrow B_1 \text{ or } B_2$ 的SDT

$B \rightarrow \{ B_1.\text{true} = B.\text{true} == \text{fall} ? \text{newlabel}() : B.\text{true}; B_1.\text{false} = \text{fall}; \} B_1 \text{ or } \{ B_2.\text{true} = B.\text{true}; B_2.\text{false} = B.\text{false}; \} B_2 \{ \text{if } B.\text{true} == \text{fall then label}(B_1.\text{true}); \}$

• 修改 $B \rightarrow B_1 \text{ and } B_2$ 的SDT

$B \rightarrow \{ B_1.\text{true} = \text{fall}; B_1.\text{false} = B.\text{false} == \text{fall} ? \text{newlabel}(): B.\text{false}; \} B_1 \text{ and } \{ B_2.\text{true} = B.\text{true}; B_2.\text{false} = B.\text{false}; \} B_2 \{ \text{if } B.\text{false} == \text{fall then label}(B_1.\text{false}); \}$

while $a < b$ **do** **if** $c < d$ **then** $x = y + z$ **else** $x = y - z$

1: (j<, a, b , 3)	1: if a < b goto 3	1: if False a < b goto 11
2: (j , - , - , 11)	2: goto 11	2:
3: (j<, c, d , 5)	3: if c < d goto 5	3: if False c < d goto 8
4: (j , - , - , 8)	4: goto	4:
5: (+ , y , z , t ₁)	5: t ₁ = y + z	5: t ₁ = y + z
6: (= , t ₁ , - , x)	6: x = t ₁	6: x = t ₁
7: (j , - , - , 1)	7: goto 1	7: goto 1
8: (- , y , z , t ₂)	8: t ₂ = y - z	8: t ₂ = y - z
9: (= , t ₂ , - , x)	9: x = t ₂	9: x = t ₂
10: (j , - , - , 1)	10: goto 1	10: goto 1
11:	11:	11:

6.4.回填

前面的问题：将标号的地址作为**继承属性**传递到生成相关跳转指令的地方，但是这样的做法需要**再进行一趟处理**，将标号和具体地址**绑定**起来

回填的基本思想：

生成一个跳转指令时，暂时不指定该跳转指令的**目标标号**。这样的指令都被放入由跳转指令组成的**列表**中。**同一个列表中的所有跳转指令具有相同的目标标号**。等到能够确定正确的目标标号时，才去填充这些指令的目标标号

B的**综合属性**(不同于前面控制流里是继承属性)：

B.truelist、**B.falselist**：指向一个包含跳转指令的列表，这些指令最终获得的目标标号就是当B为**真/假**时控制流应该转向的指令的标号

makelist(i)：创建**一个只包含i的列表**，i是跳转指令的标号，函数返回指向新创建的列表的指针

merge(p₁, p₂)：将p₁和p₂指向的列表进行**合并**，返回指向合并后的列表的指针

backpatch(p, i)：将i作为目标标号**插入**到p所指列表中的各指令中

nextquad：即将生成的**下一条指令的标号**

S.nextlist：指向一个包含跳转指令的**列表**，这些指令最终获得的目标标号就是按照运行顺序紧跟在S代码之后的指令的标号

回填技术SDT编写要点

• 语法改造

①在list**箭头指向的位置**设置**标记非终结符M**

• 在产生式末尾的语义动作中

①计算**综合属性**

②调用backpatch()函数**回填**各个list

③生成必要的附加指令

在下面的语句中，(A)不需要回填技术。

A.赋值语句 B.goto语句

C.条件语句 D.循环语句

$B \rightarrow E_1 \text{ relop } E_2 \{ B.\text{truelist} = \text{makelist}(\text{nextquad}); B.\text{falselist} = \text{makelist}(\text{nextquad}+1); \text{gen}(\text{'if' } E_1.\text{addr relop } E_2.\text{addr 'goto _'}); \text{gen}(\text{'goto _'}); \}$

$B \rightarrow \text{true} \{ B.\text{truelist} = \text{makelist}(\text{nextquad}); \text{gen}(\text{'goto _'}); \}$

$B \rightarrow \text{false} \{ B.\text{falselist} = \text{makelist}(\text{nextquad}); \text{gen}(\text{'goto _'}); \}$

$B \rightarrow (B_1) \{ B.\text{truelist} = B_1.\text{truelist}; B.\text{falselist} = B_1.\text{falselist}; \}$

$B \rightarrow \text{not } B_1 \{ B.\text{truelist} = B_1.\text{falselist}; B.\text{falselist} = B_1.\text{truelist}; \}$

$B \rightarrow B_1 \text{ or } M B_2 \{ B.\text{truelist} = \text{merge}(B_1.\text{truelist}, B_2.\text{truelist}); B.\text{falselist} = B_2.\text{falselist}; \text{backpatch}(B_1.\text{falselist}, M.\text{quad}); \}$

$B \rightarrow B_1 \text{ and } M B_2 \{ B.\text{truelist} = B_2.\text{truelist}; B.\text{falselist} = \text{merge}(B_1.\text{falselist}, B_2.\text{falselist}); \text{backpatch}(B_1.\text{truelist}, M.\text{quad}); \}$

$M \rightarrow \varepsilon \{ M.\text{quad} = \text{nextquad}; \}$

$S \rightarrow \text{if } B \text{ then } M S_1 \{ S.\text{nextlist} = \text{merge}(B.\text{falselist}, S_1.\text{nextlist}); \text{backpatch}(B.\text{truelist}, M.\text{quad}); \}$

$S \rightarrow \text{if } B \text{ then } M_1 S_1 \text{ else } M_2 S_2 \{ S.\text{nextlist} = \text{merge}(\text{merge}(S_1.\text{nextlist}, N.\text{nextlist}), S_2.\text{nextlist}); \text{backpatch}(B.\text{truelist}, M_1.\text{quad}); \text{backpatch}(B.\text{falselist}, M_2.\text{quad}); \}$

$N \rightarrow \varepsilon \{ N.\text{nextlist} = \text{makelist}(\text{nextquad}); \text{gen}(\text{'goto _'}); \}$

$S \rightarrow \text{while } M_1 B \text{ do } M_2 S_1 \{ S.\text{nextlist} = B.\text{falselist}; \text{backpatch}(S_1.\text{nextlist}, M_1.\text{quad}); \text{backpatch}(B.\text{truelist}, M_2.\text{quad}); \text{gen}(\text{'goto' } M_1.\text{quad}); \}$

$S \rightarrow S_1 M S_2 \{ S.\text{nextlist} = S_2.\text{nextlist}; \text{backpatch}(S_1.\text{nextlist}, M.\text{quad}); \}$

$S \rightarrow \text{id} = E; \mid L = E; \{ S.\text{nextlist} = \text{null}; \}$

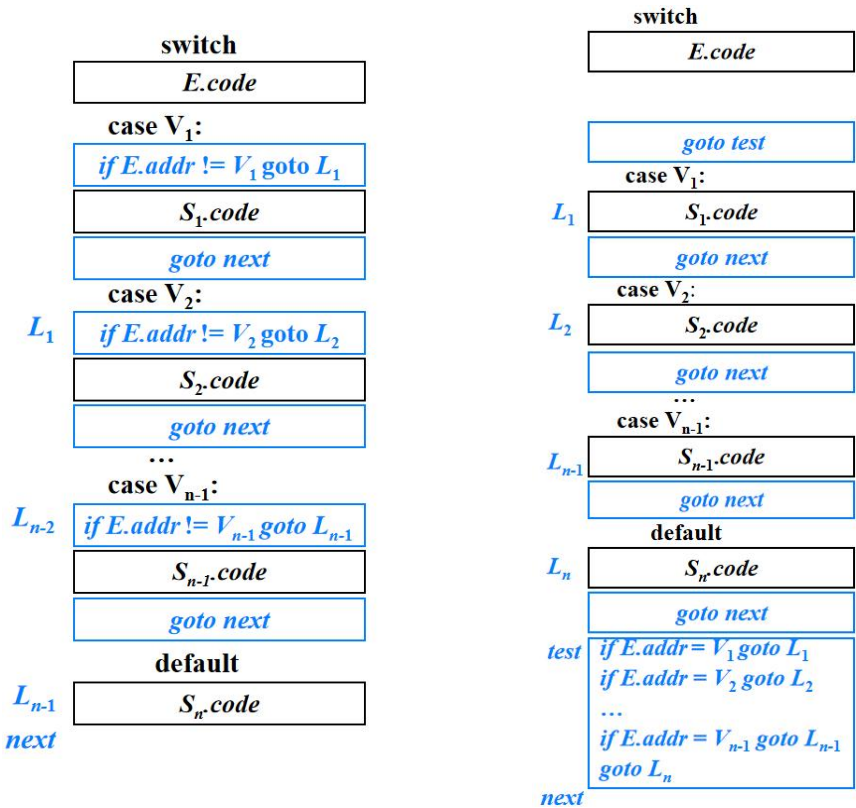
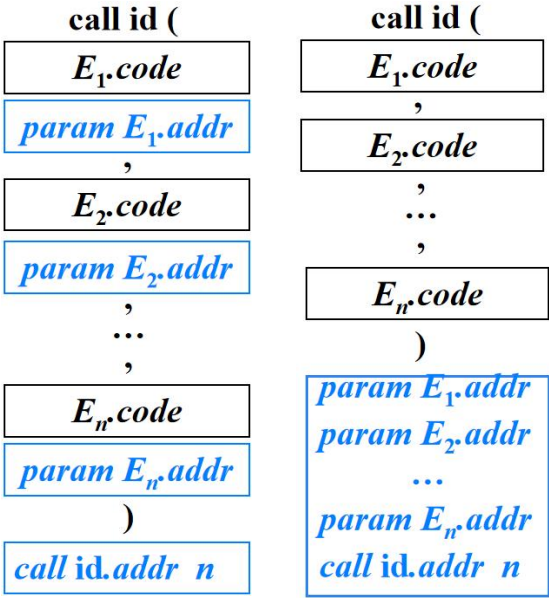
6.5.switch语句的翻译

在代码生成阶段，根据分支的个数以及这些值是否在一个较小的范围内，这种条件跳转指令序列可以被翻译成最高效的n路分支

指令 $case\ t\ V_i\ L_i$ 和 $if\ t = V_i\ goto\ L_i$ 的含义相同，但是case指令更加容易被最终的代码生成器探测到，从而对这些指令进行特殊处理

6.6.过程调用语句的翻译

$S \rightarrow call\ id\ (Elist)\{n=0;$
 $for\ q\ 中的\ 每个\ t\ do$
 $\{gen('param'\ t);n = n+1;\}$
 $gen('call'\ id.addr\ ',\ 'n);\}$
 $Elist \rightarrow E$
 $\{将q初始化为只包含E.addr;\}$
 $Elist \rightarrow Elist_1,\ E$
 $\{ 将E.addr添加到q的队尾;\}$



在回填技术中，生成一个跳转指令时，暂时不指定该跳转指令的目标标号。(√)
在回填技术中，同一个列表list中的跳转指令具有相同的目标标号。(√)
在回填技术中，同一个列表list中的跳转指令可能具有不同的目标标号。(×)
在回填技术中，等到能够确定正确的目标标号时，才去填充指令的目标标号。(√)

以下说法不正确的是(B)。
A.为布尔表达式和控制流语句生成目标代码时，关键问题之一是确定跳转指令的目标标号
B.在生成跳转指令时，就可以确定目标标号
C.在生成跳转指令时，目标标号还不能确定
D.可以将标号的地址作为继承属性传递到生成相关跳转指令的地方，但是这样的做法需要再进行一趟处理，将标号和具体地址绑定起来

7.运行存储分配

7.1存储组织

一个目标程序运行所需的存储空间主要包括：**代码区** + **数据区**

存储分配策略：**静态存储策略** + **动态存储策略**（栈式存储分配 or 堆式存储分配）

静态存储策略：对于在**编译时刻**就可以确定大小的数据对象，可以在编译时刻就为它们分配存储空间

动态存储策略：在编译时仅产生各种必要的信息，而在**运行时刻**，再动态地分配数据对象的存储空间

栈式存储分配 or 堆式存储分配

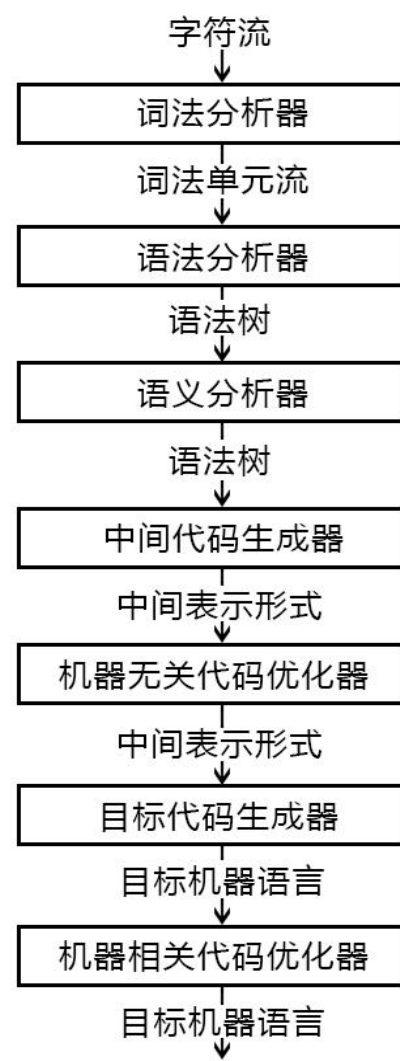
静态和动态分别对应**编译时刻**和**运行时刻**
要尽可能多的将数据对象进行静态分配，因为这些对象的地址可以被编译到目标代码中



活动记录：过程每执行一次，就为它分配一块**连续存储区**，用来管理过程一次执行所需的信息，这块连续存储区称为**活动记录**
活动：过程体的每次执行（相对过程而言）

使用**过程**（或**函数**、**方法**）作为用户自定义动作的单元的语言，其编译器通常以**过程为单位分配存储空间**

编译系统的结构



第一回

第二回

第三回

编译方法中，动态存储分配的含义是(C)。

- A.在编译阶段为源程序中的量进行分配
- B.在编译阶段为源程序中的量进行分配，运行时可动态调整
- C.在运行阶段为源程序中的量进行分配
- D.都不正确

实参	调用过程提供给被调用过程的参数
返回值	被调用过程返回给调用过程的值
控制链	指向调用者的活动记录
访问链	用来访问存放于其它活动记录中的非局部数据
保存的机器状态	通常包括返回地址和一些寄存器中的内容
局部数据	在该过程中声明的数据
临时变量	比如表达式求值过程中产生的临时变量

7.1.静态存储分配

eg.BASIC和FORTRAN

编译器为每个**过程**确定其**活动记录**在目标程序中的位置，过程**每次**执行时，它的名字都绑定到**同样的**存储单元

使用条件：①**数组上下界**必须是**常数**
②**不允许**过程的**递归**调用③**不允许**用户**动态建立**数据实体

7.1.1顺序分配法

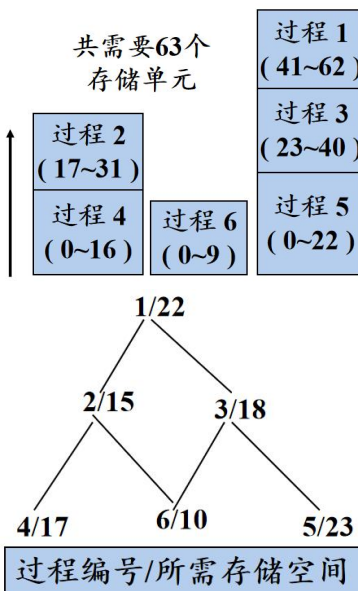
按照过程出现的先后顺序逐段分配存储空间

各过程的活动记录占用互不相交的存储空间

优点：处理上简单 缺点：对内存空间的使用不够经济合理

过程	存储区域
1	0~21
2	22~36
3	37~54
4	55~71
5	72~94
6	95~104

共需要105个存储单元



以下说法正确的是(D)。

- A.对于数据空间的存贮分配，FORTRAN采用动态贮存分配策略
- B.对于数据空间的存贮分配，C语言仅采用栈式贮存分配策略
- C.动态存储分配是指在编译阶段对源程序中的量进行分配，以使目标代码在运行时加快运行速度
- D.如果两个临时变量的作用域不相交，则可以将它们分配在同一单元中

7.2.栈式存储分配

当一个过程被**调用**时，该过程的活动记录被**压入**栈；

当过程**结束**时，该活动记录被**弹出**栈

优点：①允许活跃时段不交叠的多个过程调用之间**共享空间**②非局部变量的**相对地址总是固定的**，和过程调用序列无关

活动树：用来描述程序**运行**期间**控制**进入和离开各个活动的情况的树

当一个过程是**递归**的时候，常常会有该过程的**多个活动记录**同时出现在栈中

每个**活跃的活动**都有一个位于**控制栈**中的**活动记录**

活动树的**根**的活动记录位于**栈底**

程序控制所在的活动的记录位于**栈顶**

栈中**全部活动记录的序列**对应于在活动树中**到达当前控制所在的活动结点的路径**

PASCAL语言中过程声明的局部变量地址分配在(B)。

- A.调用者的数据区中
- B.被调用者的数据区中
- C.主程序的数据区中
- D.公共数据区中

7.1.2层次分配法

通过对过程间的**调用关系**进行分析，凡属无相互调用关系的并列过程，尽量使其局部数据**共享**存储空间

$B[n][n]$:过程调用关系矩阵, $B[i][j]=1$,表示第i个过程调用第j个过程

$Units[n]$:过程所需内存量矩阵

$base[i]$:第i个过程局部数据区的基地址

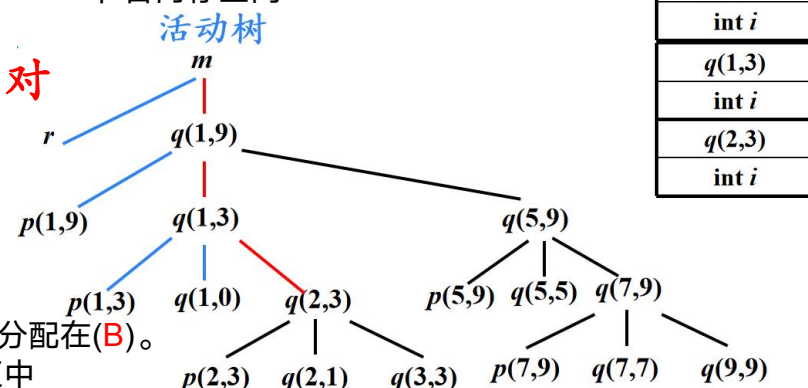
算法:从叶子节点向上遍历每个节点

$base[i]=\max(b[j]+Units[j])$, 其中

$B[i][j] = 1$, 若j不存在则 $base[i]$ 为0;

运行阶段的存储组织与管理的目的是(B)。

- A.提高编译程序的运行速度
- B.为运行阶段的存储分配做准备及提高目标程序的运行速度
- C.优化运行空间的管理
- D.节省内存空间



调用序列和返回序列

调用序列:实现**过程调用**的代码段。为一个活动记录在栈中分配空间，并在此记录的字段中填写信息

返回序列:实现**过程返回**的代码段。恢复机器状态，使得调用过程能够在调用结束之后继续执行

一个调用代码序列中的**代码**通常被**分割**到调用过程（调用者）和被调用过程（被调用者）中。返回序列也是如此

调用序列：①**调用者**计算实际参数的值②**调用者**将**返回地址**放到被调用者的机器状态字段中。将**原来的top-sp值**放到被调用者的控制链中。然后，**增加top-sp的值**，使其指向被调用者**局部数据开始的位置**③**被调用者**保存**寄存器值和其它状态信息**④**被调用者**初始化其局部数据并开始执行

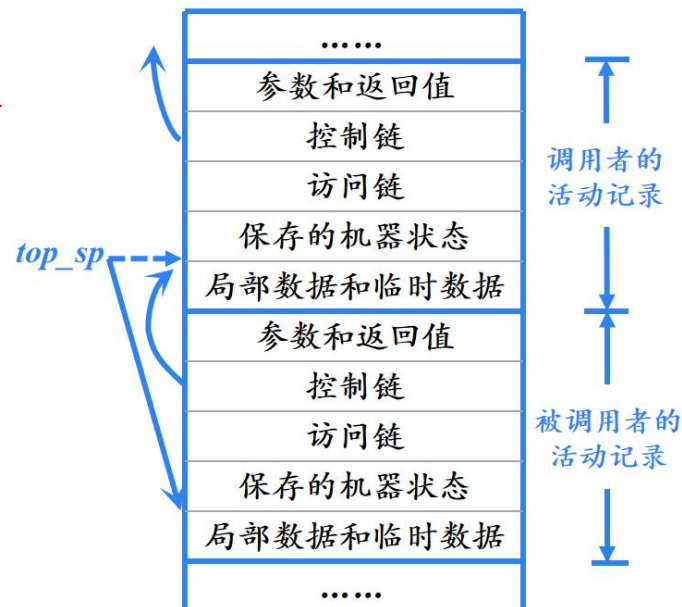
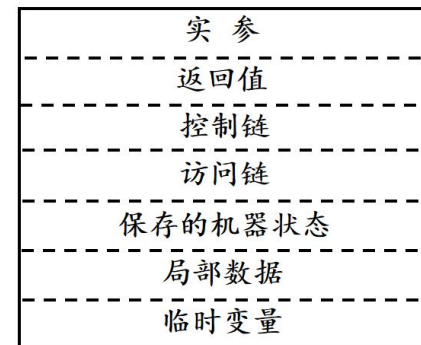
返回序列：①**被调用者**将**返回值**放到与参数相邻的位置②使用机器状态字段中的信息，**被调用者恢复top-sp和其它寄存器**，然后**跳转到**由调用者放在机器状态字段中的**返回地址**③尽管top-sp已经被减小，但**调用者**仍然知道**返回值**相对于当前top-sp值的位置。因此，调用者可以使用那个返回值

只有一个数据对象**局部于某个过程**，且当**此过程结束时它变得不可访问**，才可以使用**栈**为这个对象分配空间。尽量将对象放置在栈区而不是堆区的原因：可以**避免**对它们的空间进行**垃圾回收**，也就**减少**了相应的**开销**。

7.4.非局部数据的传递

一个过程除了可以使用**过程自身**声明的**局部数据**以外，还可以使用**过程外**声明的**非局部数据**
eg. 全局数据、外围过程定义的数据（支持过程嵌套声明的语言）

如何访问非局部数据？①**访问链**（静态链），②**display表**（嵌套层次显示表）



7.4.非局部数据的传递

访问链

静态作用域规则：只要过程 b 的声明嵌套在过程 a 的声明中，过程 b 就可以访问过程 a 中声明的对象

建立访问链的代码属于调用序列的一部分

在编译阶段通过静态分析确定嵌套深度

假设嵌套深度为 n_x 的过程 x 调用嵌套深度为 n_y 的过程 $y(x \rightarrow y)$

① $n_x < n_y$ (外层调用内层)：必定为 $n_y = n_x + 1$ ，在 y 的访问链中放置一个指向 x 的活动记录的指针

② $n_x = n_y$ (本层调用本层)：直接复制调用者的活动记录

③ $n_x > n_y$ (内层调用外层)：从 x 的活动记录开始，沿着访问链经过 $n_x - n_y + 1$ 步就可以找到离栈顶最近的 z 的活动记录。 y 的访问链必须指向 z 的这个活动记录

访问链方法存在的问题：访问外层过程名字的效率比较低

display表：一个指针数组 d

在任何时刻， $d[i]$ 均指向运行栈中最新建立的嵌套深度为 i 的过程的活动记录

(在运行栈中可能存在多个嵌套深度为 i 的过程的活动记录)

如果要访问某个嵌套深度为 i 的非局部名字 x ，只要沿着指针 $d[i]$ 找到 x 所属过程的活动记录，再根据已知的偏移量就可以在活动记录中找到 x

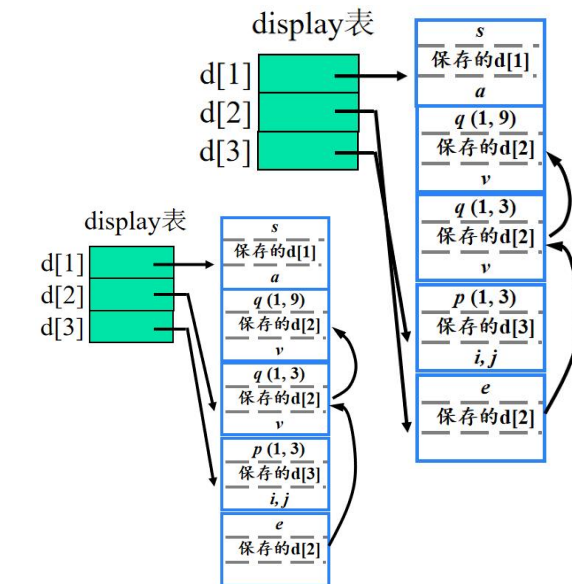
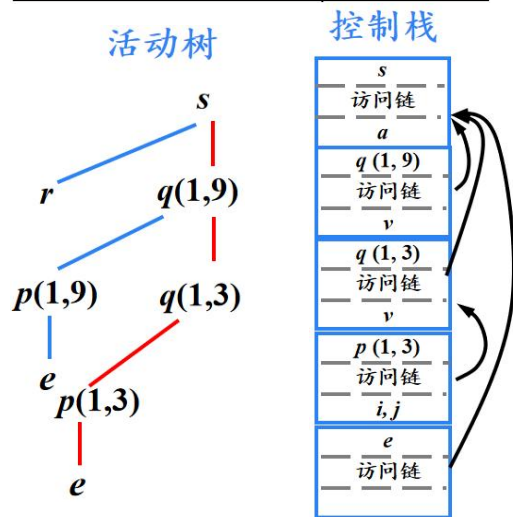
每开始一个新活动时都要修改display表，而控制从新活动返回时，必须恢复display表的状态

如果嵌套深度为 n_p 的过程 p 被调用，并且它的活动记录不是 $d[n_p]$ 直接指向的活动记录，则 p 的活动记录要保存 $d[n_p]$ 原来的值，同时置 $d[n_p]$ 指向 p 的这个活动记录

当 p 返回且它的活动记录被从运行栈中清除时，再将 $d[n_p]$ 恢复为调用 p 之前的旧值

如果运行栈中存在多个嵌套深度为 i 的过程的活动记录，则通过这些保存的 $d[i]$ 就将它们链接在了一起，但是 $d[i]$ 始终指向当前活跃的那个活动记录

过程	嵌套深度
sort	1
readarray	2
exchange	2
quicksort	2
partition	3



7.5.参数传递

形式参数：在**过程定义**中使用的参数

实际参数：在**调用过程**时使用的参数

形参和实参相关联的几种方法：传值、传地址、传值结果、传名

7.5.1传值

把实参的**值**传递给相应的形参。eg.Pascal的值参数、C/C++的缺省参数传递

实现方法：**调用过程**把**实参的值**计算出来，并传递到**被调用过程**相应的**形式单元**中

被调用过程中，像引用**局部数据**一样引用**形式参数**，**直接访问**对应的**形式单元**

7.5.2传地址

把实参的**地址**传递给相应的形参。eg.例：Pascal的var参数、C/C++的传引用

实现方法：**调用过程**把**实参的地址**传递到**被调用过程**相应的**形式单元**中

被调用过程中，对**形参**的引用或赋值被处理成对**形式单元**的**间接访问**

7.5.3传值结果

传地址的一种变形。eg.Fortran

实现方法：每个形参对应**两个形式单元**。第一个形式单元存放**实参的地址**，第二个形式单元存放**实参的值**

在过程体中，对形参的引用或赋值看作对它的第二个形式单元的**直接访问**

过程完成返回前，把第二个单元的内容存放到第一个单元所指的实参单元中

7.5.4传名

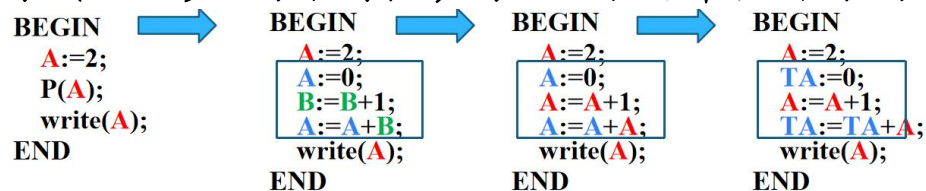
相当于把**被调用过程**的过程体**抄**到调用出现的地方，但把其中出现的**形参**都替换成相应的**实参**

实现方法：在进入**被调用过程**之前不对实参预先进行计值，而是让过程体中每当使用到相应的实参时才逐次对它实行计值（或计算地址）

通常把**实参**处理成一个子程序（称为参数子程序），每当过程体中使用到相应的实参时就调用这个子程序

eg.ALGOL60

传名方式中的**重命名问题**



...

```
procedure P(w,x,y,z);
begin
  y:=y*w;
  z:=z+x;
end
begin
  a:=5;
  b:=3;
  P(a+b,a-b,a,a);
  write(a);
end
```

传值：5

传地址：5 -> 40 -> 42

传值结果：y=40, z=7 -> a=40 -> a=7

传名：77 a = a*(a+b); -> 40 -> a= a+(a-b); -> 77

在目标代码生成阶段，符号表用于(D)。

A.目标代码生成 B.语义检查

C.语法检查 D.地址分配

7.6.符号表

符号表是用于存放**标识符的属性信息**的数据结构

种属、类型、存储位置、长度、作用域、参数和返回值信息

符号表的作用：**辅助代码生成、一致性检查**

符号表上的主要操作

声明语句的翻译（**定义性出现**）：填、查

可执行语句的翻译（**使用性出现**）：查

单个符号表的组织

方法一：一张大表

问题：不同**种属**的名字所需存放的属性信息在**数量上的差异**会造成符号表**空间的浪费**

方法二：多张子表（按种属分）变量表、数组表、过程表、…

问题：为避免**重名**问题，**插入或查找**某个符号时需要查看所有的符号表，从而造成**时间上的浪费**

解决办法：**基本属性**（直接存放在符号表中）+**扩展属性**（动态申请内存）

多个过程符号表的组织

需要考虑的问题：画过程之间的**嵌套关系**（**作用域信息**）、重名问题

常用的组织方式：每个**过程**建立一个符号表，同时需要建立起这些符号表之间的联系，

用来刻画**过程**之间的**嵌套关系**

以下说法正确的是(C)。

A.符号表由词法分析程序建立，由语法分析程序使用

B.符号表的内容在词法分析阶段填入并在以后各个阶段得到使用

C.对一般的程序设计语言而言，其编译程序的符号表应包含哪些内容及何时填入这些信息不能一概而论

D.“运算符与运算对象类型不符”属于语法错误

单个符号表：

名字	基本属性				扩展属性												
	种属	类型	地址	扩展属性指针													
abc	变量	int	0	NULL	<table><tr><td>维数</td><td colspan="2">各维维</td></tr><tr><td>长</td><td>3</td><td>4</td></tr><tr><td>2</td><td>16</td><td>4</td></tr><tr><td colspan="3">各维宽度</td></tr></table>	维数	各维维		长	3	4	2	16	4	各维宽度		
维数	各维维																
长	3	4															
2	16	4															
各维宽度																	
i	变量	int	4	NULL													
myarray	数组	int	8														
...	...																

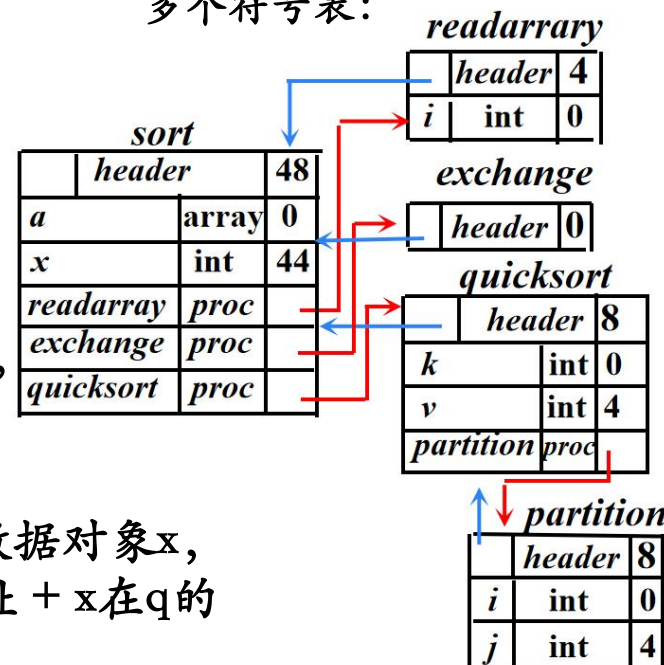
标识符基本处理方法：

case1：在**声明语句**中**定义性出现**时：以此标识符查相应于**本层的符号表①**

如果查到，则报错并发出诊断信息“**id重复声明**”②否则，在符号表中加入新登记项，将标识符及有关信息填入

case2：在**可执行语句**中**应用性出现**时：在**该层符号表**中查找该id①如果找不到，则到**直接外层符号表**中去查，如此循环，一旦找到，则在表中取出有关信息并作相应处理②如果查遍所有外层符号表均未找到该id，则报错并发出诊断信息“**id未声明**”

多个符号表：



假设过程p要访问过程q中的数据对象x，
 $x \text{ 的地址} = q \text{ 的活动记录基地址} + x \text{ 在 } q \text{ 的活动记录中的偏移地址}$

7.6.符号表

符号表的建立

嵌套过程声明语句的文法

$P \rightarrow D$

$D \rightarrow D D \mid \text{proc id} ; D S \mid \text{id} : T ;$

符号表的另一种组织方式

将所有**块**的符号表放在一个**大数组**中，然后再引入一个**块表**来描述各**块**的符号表在大数组中的位置**及其相互关系**

一个**过程**可以看作是一个**块**

display表：记录下各块所在的层号，沿着该表可以找到当前正在分析的块的各个外层

lastpar指向本过程体中**最后一个形参**在nametab中的位置

last指向本过程体中**最后一个名字**在nametab中的位置

link指向同一过程体中定义的**上一个名字**在nametab中的位置，每个过程体在nametab中登记的**第一个名字**的link为0

mktable(previous)

创建一个新的符号表，并返回指向新表的指针。参数previous指向先前创建的符号表(外围过程的符号表)

addwidth(table, width)

将table指向的符号表中所有表项的宽度之和width记录在符号表的表头中

enterproc(table, name, newtable)

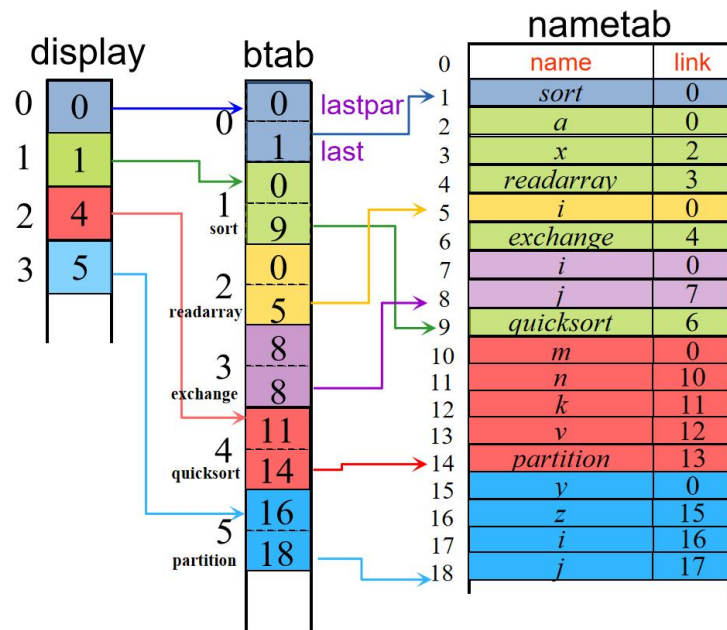
在table指向的符号表中为过程name建立一条记录，newtable指向过程name的符号表

enter(table, name, type, offset)

在table指向的符号表中为名字name建立一个新表项

offset栈：指向活跃的活动的符号表总偏移

tblptr栈：指向活跃的活动的符号表，栈顶是当前分析到的位置，栈底是活跃树的根的符号表nil



$P \rightarrow M D \{ \text{addwidth}(\text{top}(\text{tblptr}), \text{top}(\text{offset}));$
 $\text{pop}(\text{tblptr});$
 $\text{pop}(\text{offset}); \}$

$M \rightarrow \varepsilon \{ t = \text{mktable}(\text{nil});$
 $\text{push}(t, \text{tblptr});$
 $\text{push}(0, \text{offset}); \}$

$D \rightarrow D_1 D_2$

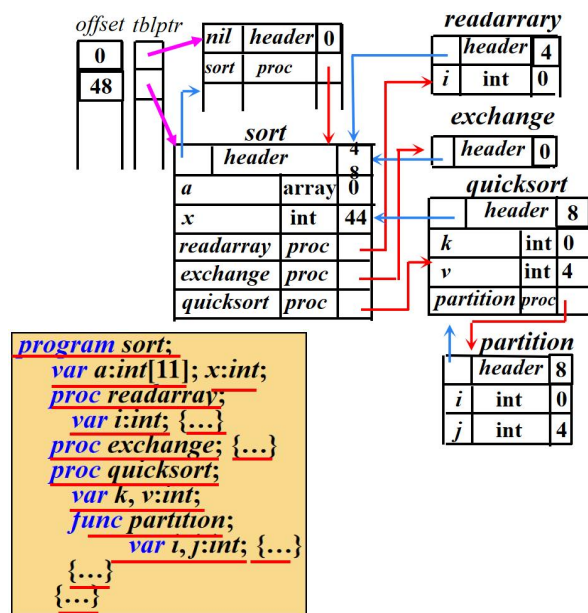
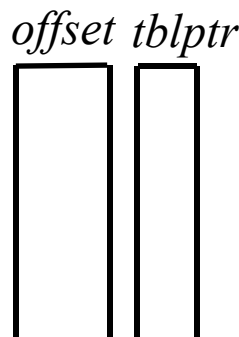
$D_p \rightarrow \text{proc id} ; N D_1 S$

$\{ t = \text{top}(\text{tblptr});$
 $\text{addwidth}(t, \text{top}(\text{offset}));$
 $\text{pop}(\text{tblptr});$
 $\text{pop}(\text{offset});$
 $\text{enterproc}(\text{top}(\text{tblptr}), \text{id.lexeme}, t); \}$

$D_v \rightarrow \text{id} : T ;$

$\{ \text{enter}(\text{top}(\text{tblptr}), \text{id.lexeme}, T.\text{type}, \text{top}(\text{offset}));$
 $\text{top}(\text{offset}) = \text{top}(\text{offset}) + T.\text{width}; \}$

$N \rightarrow \varepsilon \{ t = \text{mktable}(\text{top}(\text{tblptr}));$
 $\text{push}(t, \text{tblptr}); \text{push}(0, \text{offset}); \}$



8.代码优化

8.1流图

基本块是满足下列条件的**最大的连续三地址指令序列**

- ①控制流只能从基本块的**第一条指令**进入该块。
 - ②除了基本块的**最后一条指令**，控制流在离开基本块之前不会**跳转**或者**停机**
- 每个基本块由一组**总是一起执行**的指令组成

基本块划分算法：

首先，确定指令序列中哪些指令是**首指令**，即某个基本块的第一个指令

1. 指令序列的**第一个三地址指令**是一个首指令
2. 任意一个条件或无条件**转移指令的目标指令**是一个首指令
3. **紧跟**在一个条件或无条件**转移指令之后的指令**是一个首指令

然后，每个首指令对应的基本块包括了从它自己开始，直到**下一个首指令(不含)**或者**指令序列结尾**之间的所有指令

8.2优化的分类

机器无关优化：针对中间代码

机器相关优化：针对目标代码

局部代码优化：单个基本块范围内的优化

全局代码优化：面向多个基本块的优化

常用优化方法：①删除公共子表达式 ②删除无用代码（借助复制传播、常量合并删除无用代码）

③代码移动（利用循环不变计算） ④强度削弱（如用加代替乘，乘代替除，循环中对归纳变量进行强度削弱）

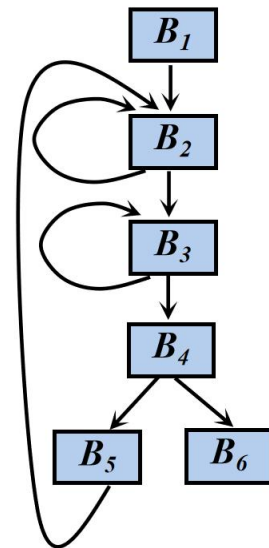
⑤删除归纳变量（删除仅用于测试的归纳变量）

优化可生成 (D) 的目标代码。
A. 运行时间较短
B. 占用存储空间较小
C. 运行时间短但占用内存空间大
D. 运行时间短且占用存储空间小

基本块内的优化为 (B)。
A. 代码外提，删除归纳变量
B. 删除多余运算，删除无用赋值
C. 强度削弱，代码外提
D. 循环展开，循环合并

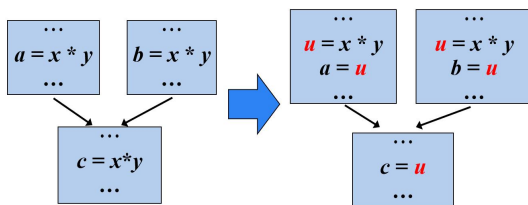
对一个基本块来说，(A)是正确的。
A. 只有一个入口语句和一个出口语句
B. 有一个入口语句和多个出口语句
C. 有多个入口语句和一个出口语句
D. 有多个入口语句和多个出口语句

流图的每个**结点**是一个**基本块**

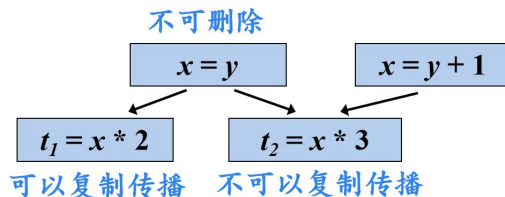


从基本块B到基本块C之间有一条边**当且仅当**基本块C的第一条指令**可能**紧跟在B的最后一条指令之后执行。此时称B是C的前驱，C是B的后继。

中间代码的优化依赖于具体的计算机 (×)
代码优化应以等价变换为基础，既不改变程序的运行结果，又能使生成的目标代码更有效 (✓)
一个程序可用一个流图来表示。 (✓)
优化工作只能在中间代码这一层次上进行。 (×)



```
for(i = 1; i < 10; i++)  
  for( n=1; n < 360/(5*i); n++)  
    { S=(5*i)/360*pi*r*r*n; ... }
```



归纳变量：对于一个变量x，如果存在一个正的或负的常数c使得每次x被赋值时它的值总增加c，那么x就称为归纳变量

8.3.基本块的优化

很多重要的局部优化技术首先把一个基本块转换成为一个**无环有向图(DAG)**

基本块的DAG表示: eg1

基于基本块的 DAG 的删除无用代码:eg2

数组元素赋值指令的表示:eg3.

基于基本块可以:

确定哪些变量的值在该基本块中**赋值前被引用**过

确定哪些语句计算的值可以在**基本块外被引用**

8.4.数据流分析

一组用来获取**有关数据如何沿着程序执行路径流动**的相关信息的技术

在每一种数据流分析应用中,都会把每个**程序点**和一个**数据流值**关联起来

数据流分析模式

语句的数据流模式: $IN[s]$ 、 $OUT[s]$ 、 f_s : 语句s的传递函数

前向传播 $OUT[s] = f_s(IN[s])$ **逆向传播** $IN[s] = f_s(OUT[s])$

基本块上的数据流模式: $IN[B]$ 、 $OUT[B]$ 、 f_B : 基本块B的传递函数

前向数据流问题: $OUT[B] = f_B(IN[B])$ 、 $f_B = f_{s1} \cdots f_{s2} \cdots f_{sn}$

逆向数据流问题: $IN[B] = f_B(OUT[B])$ 、 $f_B = f_{s1} \cdots f_{s2} \cdots f_{sn}$

数据流分析的主要应用

到达-定值分析

活跃变量分析

可用表达式分析

支配结点分析

☒ 正向	$OUT[ENTRY] = \Phi$ $OUT[B] = \Phi$	$IN[B] = \bigcup_{P \text{ 是 } B \text{ 的一个前驱}} OUT[P]$ $OUT[B] = gen_B \cup (IN[B] - kill_B)$
☒ 逆向	$IN[EXIT] = \Phi$ $IN[B] = \Phi$	$OUT[B] = \bigcup_{S \text{ 是 } B \text{ 的一个后继}} IN[S]$ $IN[B] = use_B \cup (OUT[B] - def_B)$
☑ 正向	$OUT[ENTRY] = \Phi$; $OUT[B] = U$;	$IN[B] = \bigcap_{P \text{ 是 } B \text{ 的一个前驱}} OUT[P]$ $OUT[B] = e_gen_B \cup (IN[B] - e_kill_B)$
☑ 正向	$OUT[ENTRY] = \{ENTRY\}$ $OUT[B] = N$	$IN[B] = \bigcap_{P \text{ 是 } B \text{ 的一个前驱}} OUT[P]$ $OUT[B] = IN[B] \cup \{B\}$

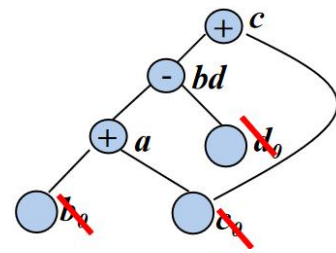
eg1.

$a = b + c$

$b = a - d$

$c = b + c$

$d = a - d$



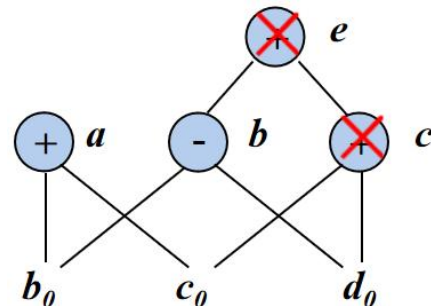
eg2.

$a = b + c$

$b = b - d$

$c = c + d$

$e = b + c$



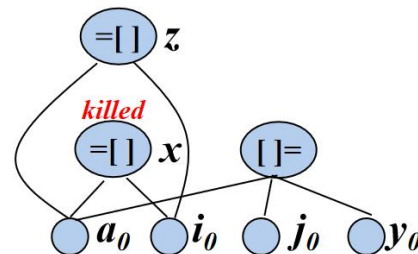
假设a和b是活跃变量,但c和e不是

eg3.

$x = a[i]$

$a[j] = y$

$z = a[i]$



以下说法不正确的是(D)。

A.大部分全局优化是基于数据流分析技术实现的

B.数据流分析是一组用来获取程序执行路径上的数据流信息的技术

C.在每一种数据流分析应用中,都会把每个程序点和一个数据流值关联起来

D.一个语句之前和之后的数据流值不受该语句的语义的约束

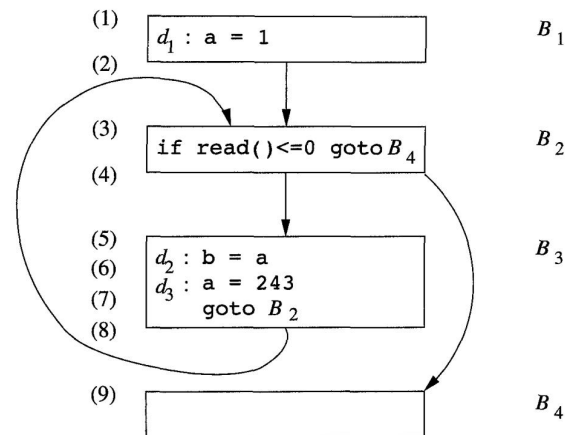
数据流分析的主要应用不包括(D)。

A.到达-定值分析

B.活跃变量分析

C.可用表达式分析

D.自然循环分析



8.4.1.到达定值分析

定值：变量 x 的**定值**是(可能)将一个值赋给 x 的**语句 d**

到达定值：如果**存在一条**从紧跟在 x 的**定值 d** 后面的**点**到达某一**程序点 p** 的**路径**，而且在此路径上 d 没有被“**杀死**”，则称**定值 d** 到达程序点 p

生成&杀死定值： $d: u = v + w$, 该语句“**生成**”了一个对变量 u 的定值 d ，并“**杀死**”了程序中其它对 u 的定值

直观理解：如果某个变量 x 的一个定值 d 到达点 p ，在点 p 处使用的 x 的值**可能**就是由 d **最后**赋予的。

分析任务：程序点 p 处使用的 x 的值可能是在哪里定值的

数据流方程：

$$OUT[ENTRY] = \Phi$$

$$OUT[B] = \Phi$$

$$IN[B] = \bigcup_{P \text{ 是 } B \text{ 的一个前驱}} OUT[P]$$

$$OUT[B] = gen_B \cup (IN[B] - kill_B)$$

被基本块 B 中各个语句杀死的定值的集合： $kill_B = kill_1 \cup kill_2 \cup \dots \cup kill_n$

基本块中没有被块中各语句“杀死”的定值的集合：

$$gen_B = gen_n \cup (gen_{n-1} - kill_n) \cup \dots \cup (gen_1 - kill_2 - kill_3 - \dots - kill_n)$$

分析结果的表示：**ud链(引用-定值链)**：是一个列表，对于变量的每一次引用，到达该引用的所有定值都在该列表中

主要用途：循环不变计算的检测、常量传播、检测“变量未经定值就被引用”

以下说法不正确的是(C)。

A. 变量 x 的定值是(可能)将一个值赋给 x 的语句

B. 假设存在一条从紧跟在 x 的定值 d 后面的点到达某一程序点 p 的路径，如果在此路径上有对变量 x 的其它定值 d' ，则称定值 d 被定值 d' “杀死”了

C. 如果存在一条从紧跟在 x 的定值 d 后面的点到达某一程序点 p 的路径，则称定值 d 到达程序点 p

D. 直观地讲，如果某个变量 x 的一个定值 d 到达点 p ，在点 p 处使用的 x 的值可能就是由 d 最后赋予的

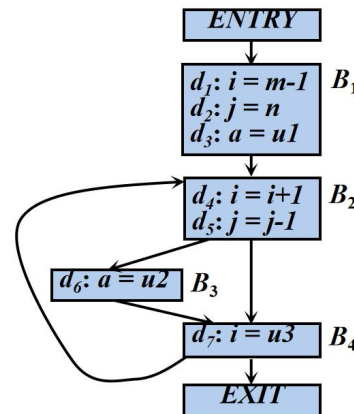
以下说法不正确的是(D)。

A. 引用-定值链(ud-chains)是一个列表，对于变量的每一次引用，到达该引用的所有定值都在该列表中

B. 如果块 B 中变量 a 的引用之前有 a 的定值，那么只有 a 的最后一次定值会在该引用的ud链中

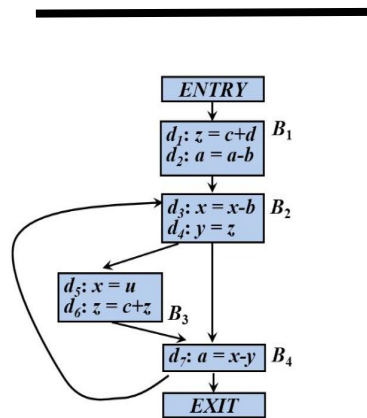
C. 如果块 B 中变量 a 的引用之前没有 a 的定值，那么 a 的这次引用的ud链就是 $IN[B]$ 中 a 的定值的集合

D. 如果块 B 中变量 a 的引用之前没有 a 的定值，那么只有 a 的最后一次定值会在该引用的ud链中



IN[B]	B ₂	B ₃	B ₄
d ₁	√	×	×
d ₂	√	×	×
d ₃	√	√	√
d ₄	×	√	√
d ₅	√	√	√
d ₆	√	√	√
d ₇	√	×	×

B	OUT[B] ⁰	IN[B] ¹	OUT[B] ¹	IN[B] ²	OUT[B] ²	IN[B] ³	OUT[B] ³
B ₁	000 0000	000 0000	111 0000	000 0000	111 0000	000 0000	111 0000
B ₂	000 0000	111 0000	001 1100	111 0111	001 1110	111 0111	001 1110
B ₃	000 0000	001 1100	000 1110	001 1110	000 1110	001 1110	000 1110
B ₄	000 0000	001 1110	001 0111	001 1110	001 0111	001 1110	001 0111
EXIT	000 0000	001 0111	001 0111	001 0111	001 0111	001 0111	001 0111



$gen_{B1} = \{d_1, d_2, d_3\}$
$kill_{B1} = \{d_4, d_5, d_6, d_7\}$
$gen_{B2} = \{d_4, d_5\}$
$kill_{B2} = \{d_1, d_2, d_7\}$
$gen_{B3} = \{d_6\}$
$kill_{B3} = \{d_3\}$
$gen_{B4} = \{d_7\}$
$kill_{B4} = \{d_1, d_4\}$

关于右图的叙述中，错误的是(B)。

A. $gen_{B1} = \{d_1, d_2\}, kill_{B1} = \{d_6, d_7\}$

B. $gen_{B2} = \{d_3, d_4\}, kill_{B2} = \{d_6, d_7\}$

C. $gen_{B3} = \{d_5, d_6\}, kill_{B3} = \{d_1, d_3\}$

D. $gen_{B4} = \{d_7\}, kill_{B4} = \{d_2\}$

8.4.2.活跃变量分析

活跃变量：对于变量x和程序点p，如果在流图中沿着从p开始的某条路径会引用变量x在p点的值，则称变量x在点p是活跃的，否则称变量x在点p不活跃

分析任务：x在程序点p处的定值在哪里会被使用

数据流方程：逆向数据流问题

$$IN[EXIT] = \Phi$$

use_B：在基本块B中引用，但是引用前在B中没有被定值的变量集合

$$IN[B] = \Phi$$

def_B：在基本块B中定值，但是定值前在B中没有被引用的变量的集合

use_B和def_B的值可以直接从流图计算出来，因此在方程中作为已知量

$$OUT[B] = \bigcup_{S \text{ 是 } B \text{ 的一个后继}} IN[S]$$

以下属于逆向数据流问题的是(B)。

$$IN[B] = use_B \cup (OUT[B] - def_B)$$

A.到达-定值分析

B.活跃变量分析

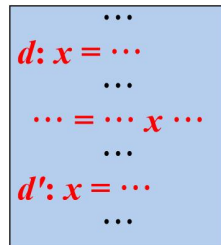
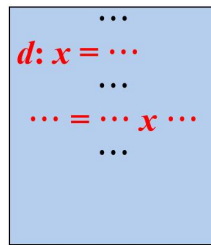
C.可用表达式分析

D.寻找支配结点

分析结果的表示：

du链(定值-引用链)：定值所有能够到达的引用的集合

主要用途：删除无用赋值、寄存器分配



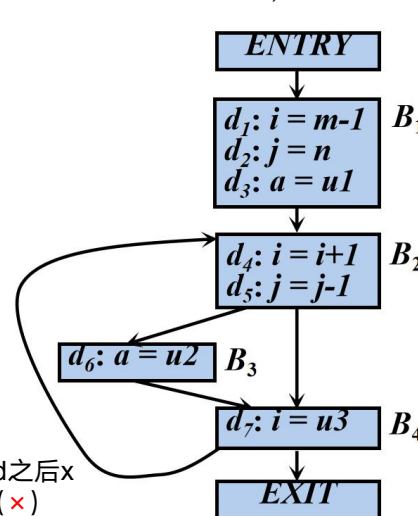
以下说法不正确的是(A)。

A.活跃变量信息可用于检测循环不变计算

B.无用赋值：如果x在点p的定值在基本块内所有后继点都不被引用，且在基本块出口之后又是不活跃的，那么x在点p的定值就是无用的

C.活跃变量信息可用于为基本块分配寄存器

D.如果所有寄存器都被占用，并且还需要申请一个寄存器，则应该考虑使用已经存放了死亡值的寄存器，因为这个值不需要保存到内存



use_{B1} = { m, n, u1 }

def_{B1} = { i, j, a }

use_{B2} = { i, j }

def_{B2} = Φ

use_{B3} = { u2 }

def_{B3} = { a }

use_{B4} = { u3 }

def_{B4} = { i }

8.4.3.可用表达式分析

如果基本块B中x的定值d之后有x的第一个定值d'，则d和d'之间x的所有引用构成d的du链(√)；如果基本块B中x的定值d之后有x的新的定值，则B中d之后x的所有引用以及OUT[B]中x的所有引用构成d的du链(×)

可用表达式：如果从流图的首节点到达程序点p的每条路径都对表达式x op y进行计算，并且从最后一个这样的计算到点p之间没有再次对x或y定值，那么表达式x op y在点p是可用的

数据流方程：

$$OUT[ENTRY] = \Phi;$$

e_gen_B：基本块B所生成的可用表达式的集合

e_kill_B：基本块B所杀死的U中的可用表达式的集合

U：所有出现在程序中一个或多个语句的右部的表达式的全集

$$OUT[B] = U;$$

将OUT集合初始化为U,主要是为了解决∩的问题，反正之后会被重新赋值

$$IN[B] = \bigcap_{P \text{ 是 } B \text{ 的一个前驱}} OUT[P]$$

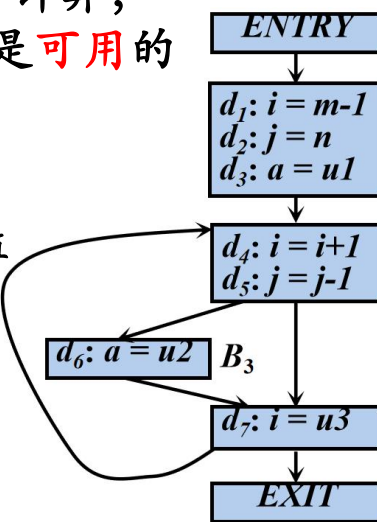
e_gen_B 计算：顺序扫描z = x op y

①把x op y加入e_gen_B ②从e_gen_B中删除和z相关的表达式

$$OUT[B] = e_{gen_B} \cup (IN[B] - e_{kill_B})$$

e_kill_B 计算：顺序扫描z = x op y

①从e_kill_B中删除表达式x op y ②把所有和z相关的表达式加入到e_kill_B中



e_gen_{B1} = { m-1 }

e_kill_{B1} = { i+1, j-1 }

e_gen_{B2} = Φ

e_kill_{B2} = { i+1, j-1 }

e_gen_{B3} = Φ

e_kill_{B3} = Φ

e_gen_{B4} = Φ

e_kill_{B4} = { i+1 }

主要用途：消除全局公共子表达式、复制传播

8.5.流图中的循环-支配结点分析

支配结点：如果从流图的入口结点到结点n的每条路径都经过结点d，则称结点d支配结点n，记为d dom n

每个结点都支配它自己

数据流方程

$OUT[ENTRY] = \{ENTRY\}$

$OUT[B] = N$

$IN[B] = \bigcap_{P \text{ 是 } B \text{ 的一个前驱}} OUT[P]$

$OUT[B] = IN[B] \cup \{B\}$

```
OUT[ENTRY]={ENTRY}
for(除ENTRY之外的每个基本块B)
  OUT[B]=N
while(某个OUT值发生了改变)
  for(除ENTRY之外的每个基本块B)
    { IN[B]=∩P是B的一个前驱 OUT[P]
      OUT[B]=IN[B] ∪ {B}
    }
```

回边：如果存在从结点n到d的有向边n→d，且d dom n，那么这条边称为回边

自然循环：满足以下性质：①有唯一的入口结点，称为首结点，首结点支配循环中的所有结点②循环中至少有一条返回首结点的路径

回边的自然循环：给定一个回边n→d，该回边的自然循环为：d，以及所有可以不经过d而到达n的结点。d为该循环的首结点

自然循环的重要性质：

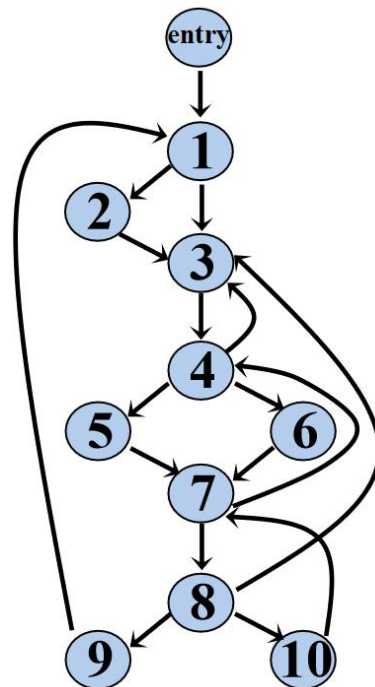
如果两个自然循环的首结点不相同，则这两个循环要么互不相交，要么一个完全包含(嵌入)在另外一个里面

如果两个循环具有相同的首结点，那么很难说哪个是最内循环。此时把两个循环合并

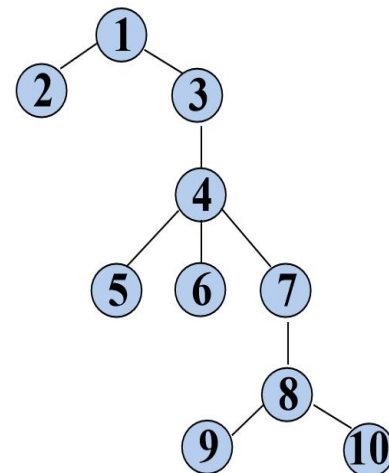
最内循环：不包含其它循环的循环

关于支配结点的二元关系，下列叙述中不正确的是(D)。

- A. 满足自反性 B. 满足传递性
- C. 满足反对称性 D. 满足对称性



支配结点	支配对象
1	1~10
2	2
3	3~10
4	4~10
5	5
6	6
7	7~10
8	8~10
9	9
10	10

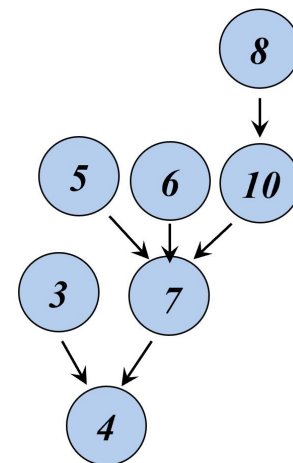


回边：4→3、7→4、8→3、9→1、10→7

算法：构造一条回边的自然循环

```
stack=Φ; loop={n,d}; push(stack,n);
while stack 不空 do
{ m=top(stack); pop(stack);
  for m的每个前驱p
  { if p不在loop中 then
    { loop=loop ∪ {p}; push(stack,p); }
  }
}
```

4→3回边的自然循环：



8.6全局优化

①删除公共子表达式

可用表达式分析的数据流问题可以帮助确定位于流图中p点的表达式是否为**全局公共子表达式**

②删除无用代码

对于**复制语句** $s: x=y$ ，如果在x的**所有**引用点都可以用对y的引用代替对x的引用(复制传播)，那么可以删除复制语句 $x=y$

复制传播的条件：复制语句 $s: x=y$ 在u点“可用”

对于每个复制语句 $x=y$ ，执行下列步骤

- ① 根据du链找出该定值所能够到达的那些对x的引用
- ② 确定是否对于**每个**这样的引用， $x=y$ 都在 $IN[B]$ 中(B 是包含这个引用的基本块)，并且 B 中该引用的前面没有 x 或者 y 的定值

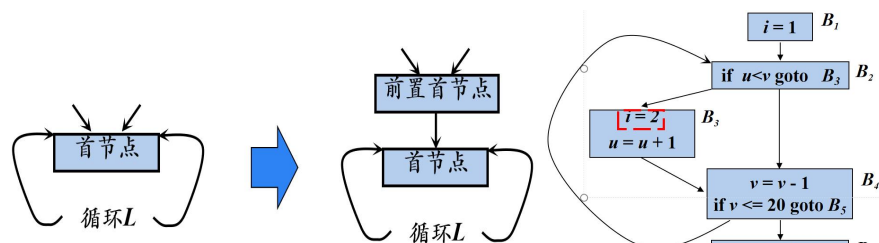
($IN[B]$ 各基本块 B 入口处的**可用复制语句集合**)

- ③ 如果 $x=y$ 满足第②步的条件，删除 $x=y$ ，且把步骤①中找到的对 x 的引用用 y 代替

③代码移动

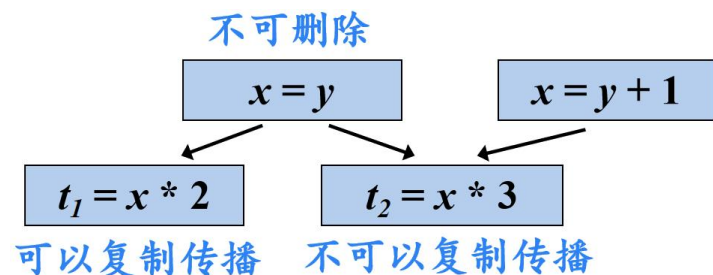
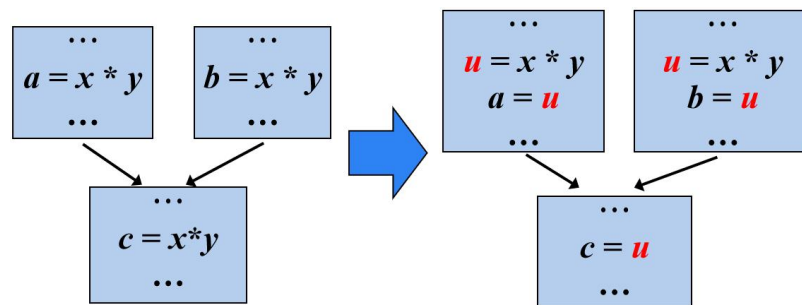
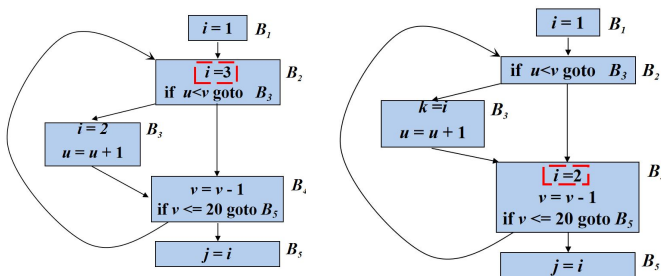
循环不变计算的检测：见右侧算法

代码外提：将循环不变计算将被移至首结点之前，称为**前置首节点**



$s: x = y + z$ 移动的条件：

- ① s 所在的基本块是循环所有**出口结点**(有后继结点在循环外的结点)的**支配结点**
- ② 循环中**没有其它语句对 x 赋值**
- ③ 循环中**对 x 的引用仅由 s 到达**



对于语句 $s: z = x \text{ op } y$ ，如果 $x \text{ op } y$ 在 s 之前可用，那么执行如下步骤：

- ① 从 s 开始逆向搜索，但不穿过任何计算了 $x \text{ op } y$ 的块，找到所有离 s 最近的计算了 $x \text{ op } y$ 的语句
- ② 建立新的临时变量 u
- ③ 把步骤①中找到的语句 $w = x \text{ op } y$ 用 $u = x \text{ op } y$ ， $w = u$ 替代
- ④ 用 $z = u$ 替代 s

循环不变计算的检测

输入：循环 L ，每个三地址指令的 ud 链

输出： L 的循环不变计算语句

方法

1. 将下面这样的语句标记为“不变”：语句的各运算分量或者是**常数**，或者其**所有定值点都在循环 L 外部**
2. 重复执行步骤(3)，直到某次没有新的语句可标记为“不变”为止
3. 将下面这样的语句标记为“不变”：先前没有被标记过，且各运算分量或者是**常数**，或者其**所有定值点都在循环 L 外部**，或者**只有一个到达定值，该定值是循环中已经被标记为“不变”的语句**

8.6全局优化

③代码移动

代码移动算法

输入：循环 L 、 ud 链、支配结点信息

输出：修改后的循环

方法：

1. 寻找循环不变计算

2. 对于步骤(1)中找到的每个循环不变计算，检查是否满足上面的三个条件

3. 按照循环不变计算找出的次序，把所找到的满足上述条件的循环不变计算外提到前置首结点中。如果循环不变计算有分量在循环中定值，只有将定值点外提后，该循环不变计算才可以外提

④强度削弱

基本归纳变量： $i = i + c$

归纳变量： $j = c \times i + d$ ， j 属于 i 族，与 j 相关联的三元组是 (i, c, d)

输入：带有循环不变计算信息和到达定值信息的循环 L

输出：一组归纳变量（三元组的形式）

方法：

1. 扫描 L 的语句，找出所有基本归纳变量。

2. 寻找 L 中只有一次定值的变量 k ，满足 $k = c' \times j + d'$ ， j 是基本的或非基本的归纳变量

如果 j 是基本归纳变量，那么 k 属于 j 族。 k 对应的三元组可以通过其定值语句确定。

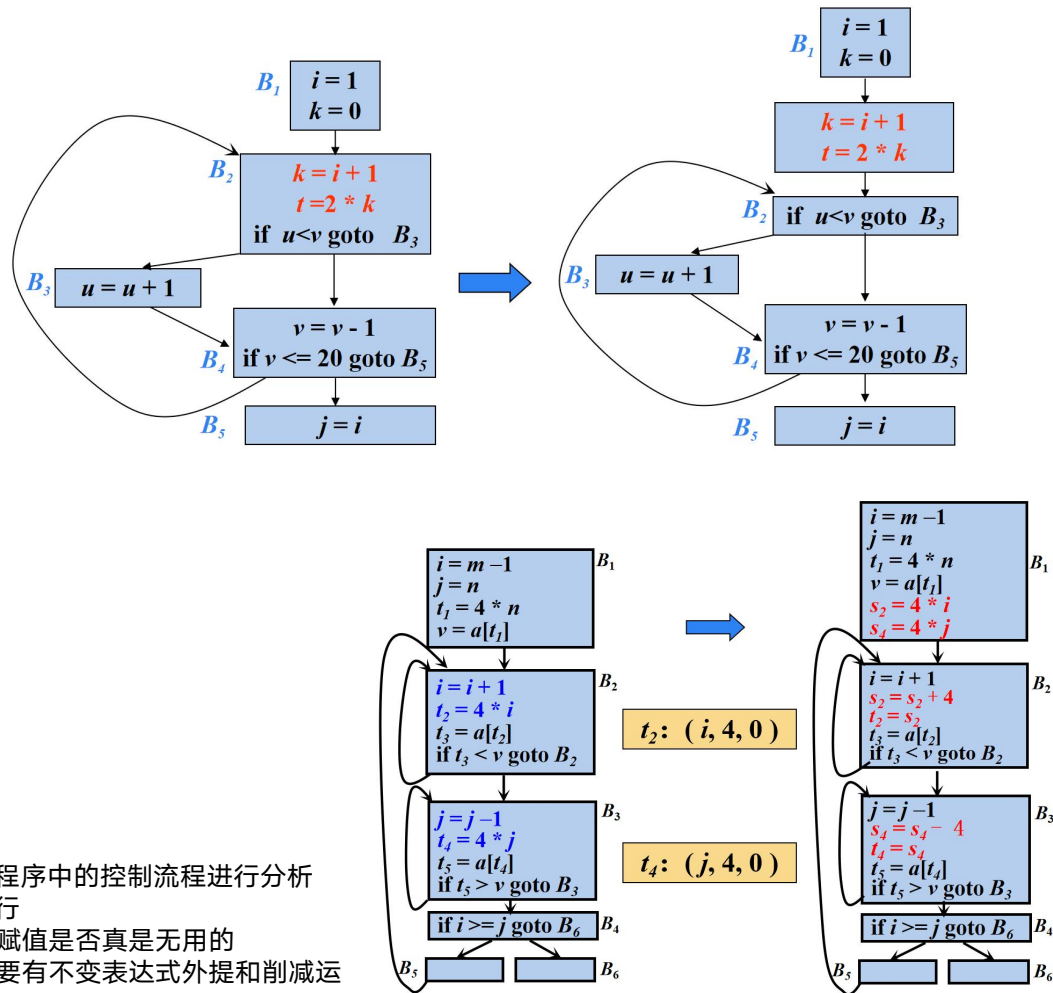
如果 j 不是基本归纳变量，假设其属于 i 族， k 的三元组可以通过 j 的三元组和 k 的定值语句来计算，此时我们还要求：循环 L 中对 j 的唯一定值和对 k 的定值之间没有对 i 的定值
循环 L 外没有 j 的定值可以到达 k （这两个条件是为了保证对 k 进行赋值的时候， j 当时的值一定等于 $c \times (i \text{ 当时的值}) + d$ ）

$s: x = y + z$ 移动的条件：

① s 所在的基本块是循环所有出口结点(有后继结点在循环外的结点)的支配结点

② 循环中没有其它语句对 x 赋值

③ 循环中对 x 的引用仅由 s 到达



以下说法不正确的是(B)。

A. 为了找出程序中的循环，就需要对程序中的控制流程进行分析

B. 在循环中强度削弱仅对乘法运算实行

C. 仅考虑一个基本块，不能确定一个赋值是否真是无用的

D. 在中间代码优化中循环上的优化主要有不变表达式外提和削减运算强度

输入：带有到达定值信息和已计算出的归纳变量族的循环 L

输出：修改后的循环

方法：对于每个基本归纳变量 i ，对其族中的每个归纳变量 j ： (i, c, d)

执行下列步骤

1. 建立新的临时变量 t 。如果变量 j_1 和 j_2 具有相同的三元组，则只为它们建立一个新变量
2. 在前置节点的末尾，添加语句 $t = c \times i$ 和 $t = t + d$ ，使得在循环开始的时候 $t = c \times i + d$
3. 在 L 中紧跟定值 $i = i + n$ 之后，添加 $t = t + c \times n$ 。将 t 放入 i 族，其三元组为 (i, c, d)
4. 用 t 代替对 j 的赋值

8.6全局优化

⑤删除归纳变量（删除仅用于测试的归纳变量）

对于在**强度削弱**算法中引入的复制语句 $j=t$ ，如果在**归纳变量j**的所有引用点都可以用对t的引用**代替**对j的引用，并且**j在循环的出口处不活跃**，则可以删除复制语句 $j=t$

强度削弱后，有些归纳变量的作用**只是用于测试**。如果可以用对**其它归纳变量**的测试**代替**对这种归纳变量的测试，那么可以删除这种归纳变量

eg1. (relop i x B) 替换为 (relop j c*x+d B)

eg2. (relop $i_1 i_2 B$) 且有 $j_1(i_1, c, d)$ 和 $j_2(i_2, c, d)$ 时，那么可以将其替换为 (relop $j_1 j_2 B$)

删除公共子表达式需要用到以下哪种数据流分析技术(C)。

- A. 到达定值分析
- B. 活跃变量分析
- C. 可用表达式分析
- D. 以上都不正确

复制传播需要用到以下哪种数据流分析技术(C)。

- A. 到达定值分析
- B. 活跃变量分析
- C. 可用表达式分析
- D. 以上都不正确

删除无用代码需要用到以下哪种数据流分析技术(B)。

- A. 到达定值分析
- B. 活跃变量分析
- C. 可用表达式分析
- D. 以上都不正确

识别循环不变计算需要用到以下哪种数据流分析技术(A)。

- A. 到达定值分析
- B. 活跃变量分析
- C. 可用表达式分析
- D. 以上都不正确

采用DAG图不可以实现的优化有(D)。

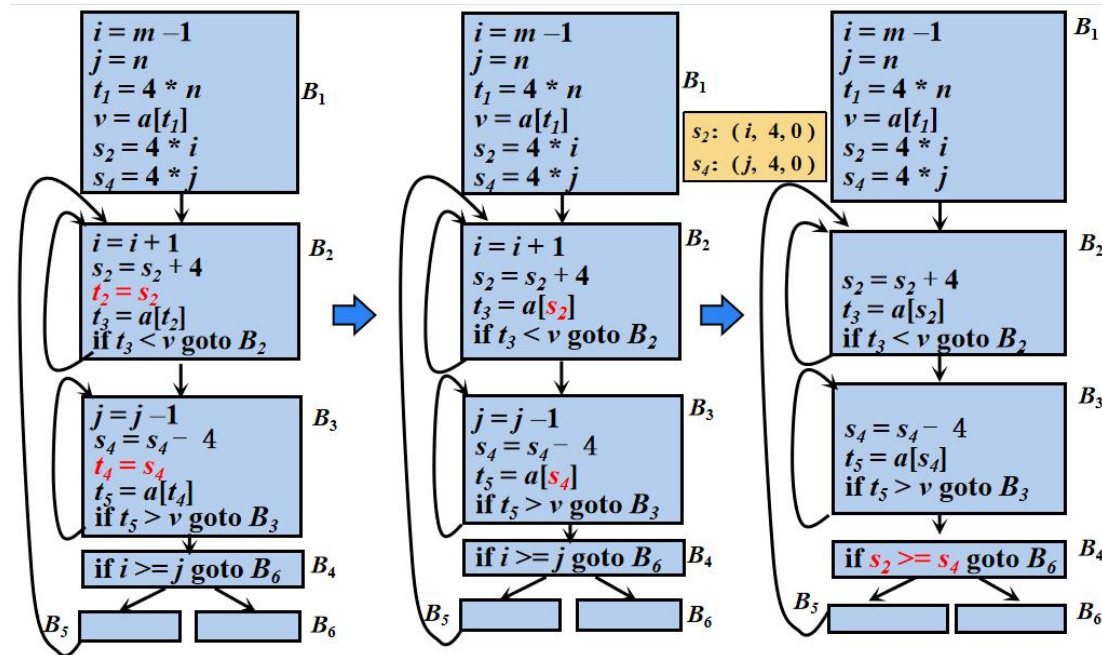
- A. 合并已知量
- B. 删除公共子表达式
- C. 删除无用赋值
- D. 删除归纳变量

下列优化方法(C)不是针对循环优化进行的。

- A. 强度削弱
- B. 删除归纳变量
- C. 删除多余运算
- D. 代码外提

(B)属于局部优化。

- A. 代码外提
- B. 删除多余运算
- C. 强度削弱
- D. 删除归纳变量



程序流图中，若m是n的必经节点，并且存在有向边 $n \rightarrow m$ ，则称 $n \rightarrow m$ 是流图中的(B)

- A. 一个循环
- B. 一条回边
- C. 一条基本边
- D. 一个基本通图

以下说法正确的是(A)。

- A. 进行代码优化时要着重考虑循环的代码优化，这对提高目标代码的效率将起更大作用
- B. 强度削弱是局部优化的一种方法
- C. 循环不变计算都可以移动到循环之前
- D. 循环中的无用赋值在循环优化时均可删除

以下说法不正确的是(A)。

- A. 如果从流图的首节点到达程序点p的每条路径都对表达式 $x \text{ op } y$ 进行计算，那么表达式 $x \text{ op } y$ 在点p是可用的
- B. 在x的引用点u可以用y代替x的条件是：复制语句 $x = y$ 在引用点u处可用
- C. 在x的引用点u可以用y代替x的条件是：从流图的首节点到达u的每条路径都存在复制语句 $x = y$ ，并且从最后一条复制语句 $x = y$ 到点u之间没有再次对x或y定值
- D. 如果一个值在基本块结尾处是死的就不必在结尾处保存这个值

9.代码生成

9.1代码生成器的主要任务

指令选择: 选择适当的**目标机指令**来实现中间表示(IR)语句

寄存器分配和指派: 把哪个值放在哪个寄存器中

指令排序: 按照什么顺序来安排指令的执行

9.2一个简单的目标机模型

三地址机器模型: 加载、保存、运算、跳转等操作; 内存按**字节**寻址; n 个通用寄存器 $R_0, R_1, \dots, R_{n-1}$; 假设所有的运算分量都是**整数**; 指令之间可能有一个**标号**

主要指令: 加载LD, 保存ST, 运算OP, 无条件跳转BR, 条件跳转Bcond

寻址模式: 选择a为变量名, c为常数, r为寄存器, 7中寻址方式见地址寻址图

运算语句的目标代码

三地址语句: $x = y - z$

目标代码:

```
LD R1, y // R1 = y
LD R2, z // R2 = z
SUB R1, R1, R2 // R1 = R1 - R2
ST x, R1 // x = R1
```

数组寻址语句的目标代码

三地址语句

$a[j] = c$

a 是一个实数数组, 每个实数占8个字节

目标代码

```
LD R1, c // R1 = c
LD R2, j // R2 = j
ST a(R2), R1 // contents(a+contents(R2))=R1
```

数组寻址语句的目标代码

三地址语句

$b = a[i]$

a 是一个实数数组, 每个实数占8个字节

目标代码

```
LD R1, i // R1 = i
LD R2, a(R1) // R2=contents ( a + contents(R1) )
ST b, R2 // b = R2
```

指针存取语句的目标代码

三地址语句

$x = *p$

目标代码

```
LD R1, p // R1 = p
LD R2, 0(R1) // R2 = contents ( 0 + contents (R1) )
ST x, R2 // x = R2
```

经编译得到的目标程序是(A)。

A.机器语言程序或汇编语言程序

B.四元式序列

C.三元式序列

D.二元式序列

	例	效果
a	$LD R1, a$	$R1 = contents (a)$
r	$LD R1, R2$	$R1 = contents (R2)$
$a(r)$	$LD R1, a(R2)$	$R1 = contents (a + contents(R2))$
$c(r)$	$LD R1, 100 (R2)$	$R1 = contents (100 + contents(R2))$
$*c(r)$	$LD R1, *100(R2)$	$R1 = contents (contents (100 +contents(R2)))$
$*r$	$LD R1, * R2$	$R1 = contents (contents (contents (R2)))$
$\#c$	$LD R1, \#100$	$R1 = 100$

(A)不可能是目标代码。

A.中间代码

B.汇编代码

C.绝对指令代码

D.可重定位指令代码

过程调用和返回:

静态/栈式内存分配的方式

callee.staticArea: callee的活动

记录在静态区中的起始位置

callee.codeArea: callee的目标代

码在代码区中的起始位置

#here + 20/here + 16: 返回地

址

#caller.recordsize: callee活动记

录的大小。

三地址语句

$if x < y goto L$

目标代码

```
LD R1, x // R1 = x
LD R2, y // R2 = y
SUB R1, R1, R2 // R1=R1 - R2
BLTZ R1, M // if R1 < 0 jump to M
```

三地址语句

call callee

目标代码

(静态方式)

目标代码

(栈式)

```
ST callee.staticArea, #here + 20
BR callee.codeArea

ADD SP, SP, #caller.recordsize
ST 0(SP), #here + 16
BR callee.codeArea
```

BR *callee.staticArea

➤ 被调用过程

BR *0(SP)

➤ 调用过程

SUB SP, SP, #caller.recordsize

9.4寄存器的选择

寄存器描述符：记录每个寄存器当前存放的是哪些变量的值

地址描述符：记录运行时每个名字的当前值存放在哪个或哪些位置，可以存放在对应的符号表条目中

基本块的**收尾处理**：①**忘记**只在基本块**内部**使用的**临时变量**②**存放**出口处可能**活跃**的变量

管理寄存器和地址描述符：当生成加载、保存和其他指令时，必须同时**更新寄存器和地址描述符**

“LD R, x”：①使R的寄存器描述符只包含x②x的地址描述符新增R③不同于x的地址描述符删除R

“OP R_x, R_y, R_z ”：①使 R_x 的寄存器描述符只包含x②不同于 R_x 的寄存器描述符中删除x③使x的地址描述符只包含 R_x ④不同于x的地址描述符删除 R_x

“ST x, R”①x的地址描述符新增自己的内存位置

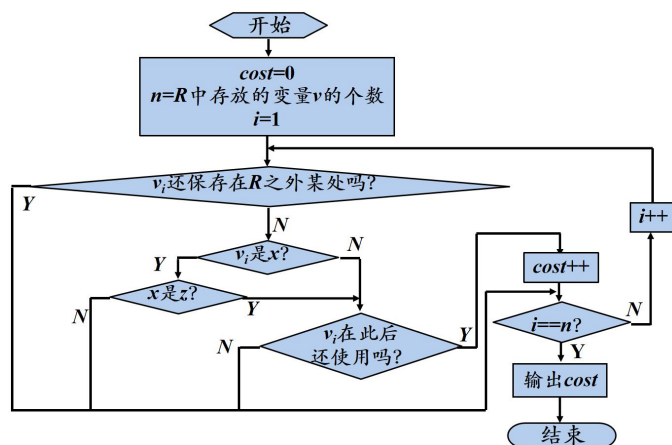
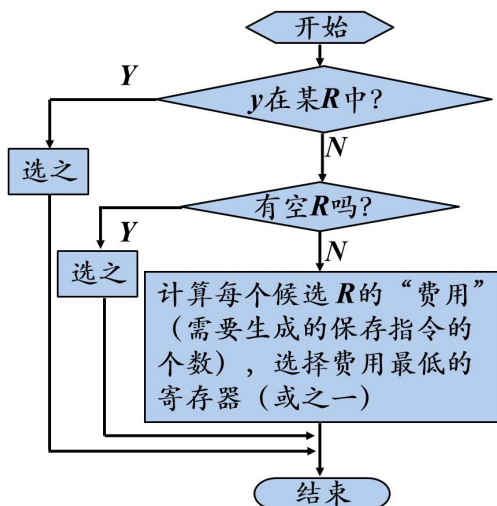
复制语句 $x=y$ ：① R_y 的寄存器描述符增加x②使x的地址描述符只包含 R_y （如果需生成加载指令“LD R_y, y' ”）

获取 $x=y \text{ op } z$ 的三地址指令I：

①getReg(I)：为x、y、z选择寄存器②判断是否需要LD R_y, y' ，LD R_z, z ③OP R_x, R_y, R_z

getReg()选择寄存器 R_y ：

计算R的费用：



选择方法与 R_y 类似，区别之处在于：

①**只存放了x**的值的寄存器对 R_x 来说总是可接受的，即使x就是y或z之一

②如果y在指令I之后**不再使用**，且(在必要时加载y之后) R_y **仅仅保存了y的值**，那么， R_y 同时也可以用作 R_x 。对z和 R_z 也有类似选择

当I是复制指令 $x=y$ 时，选择好 R_y 后，令 $R_x = R_y$

9.5 窥孔优化

相当于维持一个**滑动窗口**，如果满足某些条件，就用更快或者更短的指令来**替换**窗口中的指令序列

可以用于机器相关优化，也可以在**中间代码生成之后**直接应用窥孔优化来提高中间表示形式的质量

① 冗余指令删除

消除冗余的加载和保存指令(有标号不能删除，因为可能从其他地方跳转过来)

消除不可达代码 eg. 一个紧跟在**无条件跳转之后**的**不带标号**的指令可以被删除

② 控制流优化

在代码中出现**跳转到跳转指令的指令**时，某些条件下可以使用**一个跳转指令**来代替

③ 代数优化

代数恒等式：消除窥孔中类似于 $x=x+0$ 或 $x=x*1$ 的运算指令

强度削弱：对于乘数(除数)是**2的幂**的定点数乘法(除法)，用**移位运算**实现代价比较低

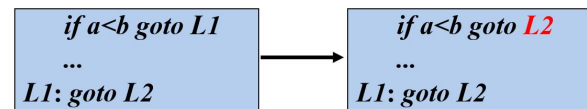
除数为常量的浮点数**除法**可以通过乘数为该常量倒数的**乘法**来求近似值

④ 机器特有指令的使用

充分利用目标系统的某些**高效的特殊指令**来提高代码效率

eg. INC指令可以用来替代加1的操作

三地址指令序列	目标代码
➤ $a=b+c$	➤ LD R0, b // $R0=b$
➤ $d=a+e$	➤ ADD R0, R0, c // $R0=R0+c$
	➤ ST a, R0 // $a=R0$
	➤ LD R0, a // $R0=a$
	➤ ADD R0, R0, e // $R0=R0+e$
	➤ ST d, R0 // $d=R0$



以下说法不正确的是(D)。

A.源程序到目标程序的变换是等价变换，即两者结构不同，但语义是一致的

B.源程序和目标程序是等价关系

C.目标代码生成时，需要考虑目标计算机的指令系统

D.窥孔优化是在中间代码层次上进行的优化工作

所有编译程序都有目标代码生成阶段。(✓)

代码生成器的设计要着重考虑目标代码的质量问题。(✓)

目标代码生成时，无需考虑目标计算机的系统结构。(✗)

谢谢大家！

附：自己写的没什么用的笔记
(<https://www.aliyundrive.com/s/UGZtf55NAsu>)